



Advanced Debugging Techniques Using MPLAB[®] X and XC Compilers

Cindy Otto, SW Principal ESE
John Haroian, NW Sr. Staff ESE



Objectives

- **When you walk out of this class, you will know how to demonstrate various MPLAB® X XC Compiler Tips and Tricks: C Startup, placing a function/variable at a given memory location.**
- **You will be able to demonstrate tools: Call Graph, Stack Guidance, Call Stack, Compiler Advisor, Code Coverage.**
- **You will be able to use MPLAB® X IDE ARM tools for a better debug experience. You will learn to use multiple methods of logging to determine the internal state of the MCU, debug a live target, and trace back to source cause of an exception using these tools.**

Agenda

- **MPLAB[®] XC Specifics**
 - C Startup
 - Placing a variable/function at a specific address
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
- **Other Tools**
 - Compiler Advisor
 - Analysis Tool Suite-Code Coverage
- **MPLAB[®] X IDE Arm[®] Tools**
 - printf/logging without using a UART
 - Live Connect
 - Trace back to Source

Agenda

- **MPLAB[®] XC Specifics**

- **C Startup**
- Placing a variable/function at a specific address
- Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack

- **Other Tools**

- Compiler Advisor
- Analysis Tool Suite-Code Coverage

- **MPLAB[®] X IDE Arm[®] Tools**

- printf/logging without using a UART
- Live Connect
- Trace back to Source

The C startup

- Lives between reset and main
- Initializes stack
- Initializes various critical SFRs, depending on the architecture
- Calls user initialization function(s)
- May run at a reduced clock rate
- Can be *carefully* modified
- Startup initializes C runtime environment
 - Initializes or clears global/static variables & calls C++ constructors

MPLAB® X XC8 PIC® MCU Startup

- **Startup code is generated**
 - Code is placed in `startup.s`
 - Generated code can be optionally saved and not regenerated
 - Can optionally choose not to link this file into the program
 - Reset condition flags can be optionally backed up(except for PIC18)
 - Not intended to be modified
- **`powerup.S`**
 - Customized copy included, if present
 - Executed right after reset
 - Located in `<xc8 dir>/<version>/pic/sources/c9x/pic`
 - Copy this file into your project directory
 - Edit `powerup.S` as needed
 - Make sure “Preprocess assembly files” is checked in Project Properties>XC8 Compiler>Preprocessing and messages

Oscillator Startup on 8-bit PIC[®] MCUs

- Many 8-bit devices start at 500 kHz
- Datasheet is your friend! Check:
 - Operation
 - Configuration bits
 - OSCCON register
- **`#define _XTAL_FREQ`** for delay functions

MPLAB® X XC8 AVR® MCU Startup

Table 4-8. Runtime Startup Code Sections used Before main

Section name	Description
<code>.init0</code>	Weakly bound to <code>__init()</code> , see 4.9.3. The Powerup Routine . If user defines <code>__init()</code> , it will be jumped into immediately after a reset.
<code>.init1</code>	Unused. User definable.
<code>.init2</code>	In C programs, weakly bound to code which initializes the stack and clears <code>__zero_reg__</code> (r1).
<code>.init3</code>	Unused. User definable.
<code>.init4</code>	This section contains the code from <code>libgcc.a</code> that copies the contents of <code>.data</code> from the program to data memory, as well as the code to clear the <code>.bss</code> section.
<code>.init5</code>	Unused. User definable.
<code>.init6</code>	Unused for C programs.
<code>.init7</code>	Unused. User definable.
<code>.init8</code>	Unused. User definable.
<code>.init9</code>	Calls the <code>main()</code> function.

MPLAB® X XC8 AVR® Customized MCU Startup

```
#include <avr/io.h>
```

```
.section .init1,"ax",@progbits
```

```
ldi r16, 0xff
```

```
out _SFR_IO_ADDR(PORTB), r16
```

```
out _SFR_IO_ADDR(DDRB), r16
```

- Place routine in an assembly source file and include in project
- Make sure “Preprocess assembly files” is checked in Project Properties>XC8 Compiler>Preprocessing and messages

MPLAB® X XC8 AVR® Customized MCU Startup

```
#include <avr/io.h>
```

```
void my_init_portb (void) __attribute__((naked))  
    __attribute__((section (".init3")));
```

```
void my_init_portb (void) {  
    PORTB = 0xff;  
    DDRB  = 0xff;  
}
```

MPLAB® X XC-DSC Customized Startup

- **Initializes C runtime environment(initializes SP, SPLIM, clears uninitialized objects and loads initialized objects with data stored in program memory)**
- **Calls any C function with the user_init attribute**
- **Calls main() after all user_init functions**
- **If main() returns, debug break or reset**

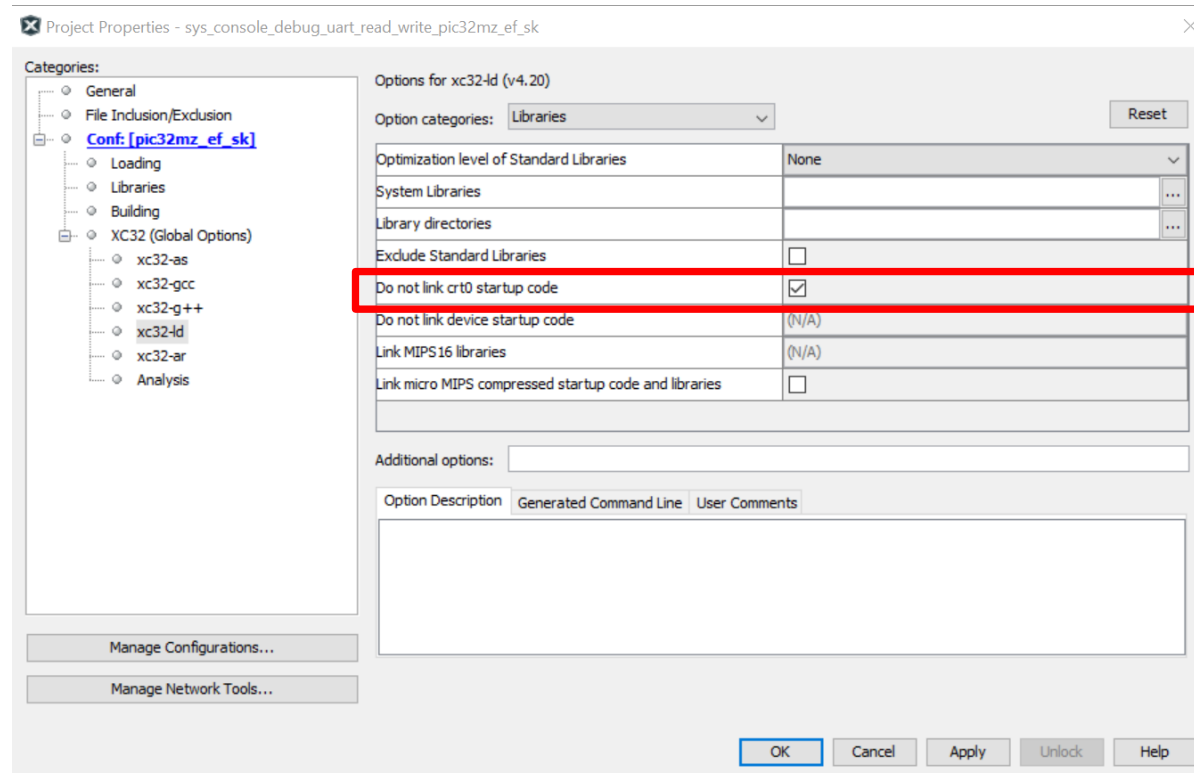
MPLAB® X XC32 Startup

- Initializes stack pointer, heap, ECC SRAM(if available)
- If `_on_reset()` is defined, call it
- Initialize data, clear uninitialized data
- If `_on_bootstrap()` is defined, call it
- Call `main()`
- If `main()` returns, debug break or reset
- To add custom code to this, create a void function called `_on_reset` and/or `_on_bootstrap` in one of your application files.

Using Customized Startup Code instead of default

Linking a custom startup code for PIC32M based projects:

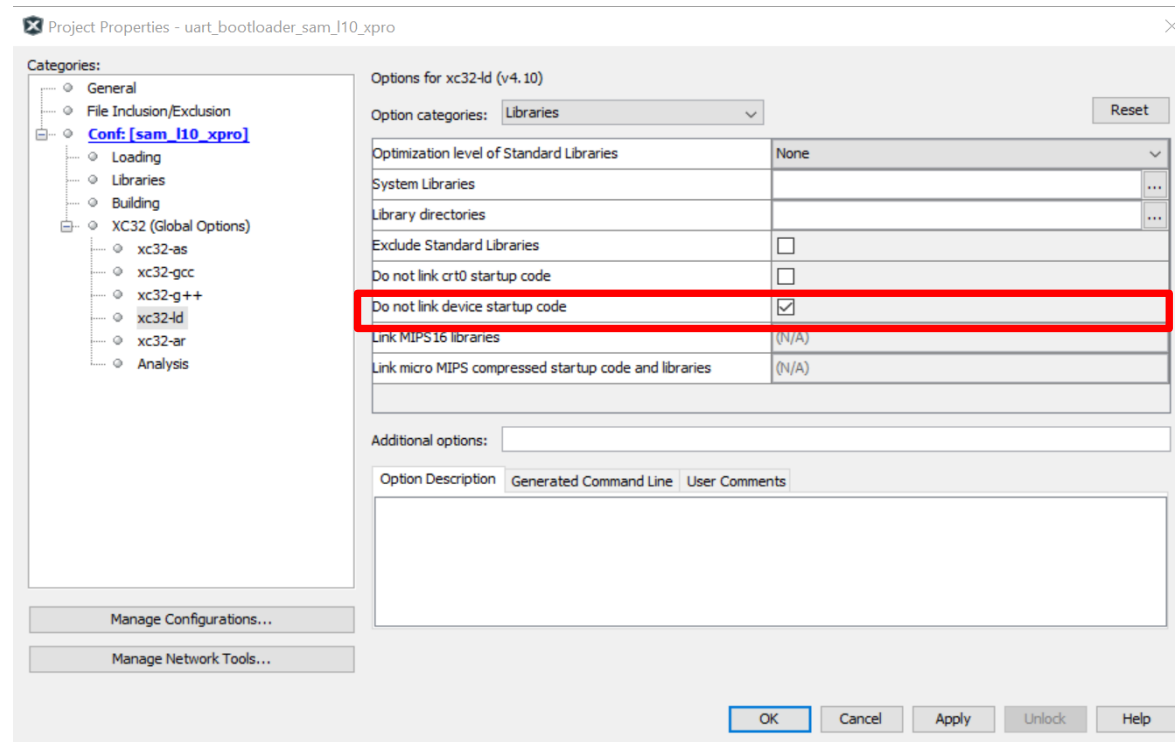
1. Copy the Startup code file from “pic32mx\lib\crt0.S file” to the project directory.
2. Customize the Startup code.
3. In MPLAB® X IDE select “Do not link crt0 startup code”
4. Add the modified startup code to MPLAB® X IDE “Source Files” tab.



Using Customized Startup Code instead of default

Linking a custom startup code for PIC32C/Arm® based projects:

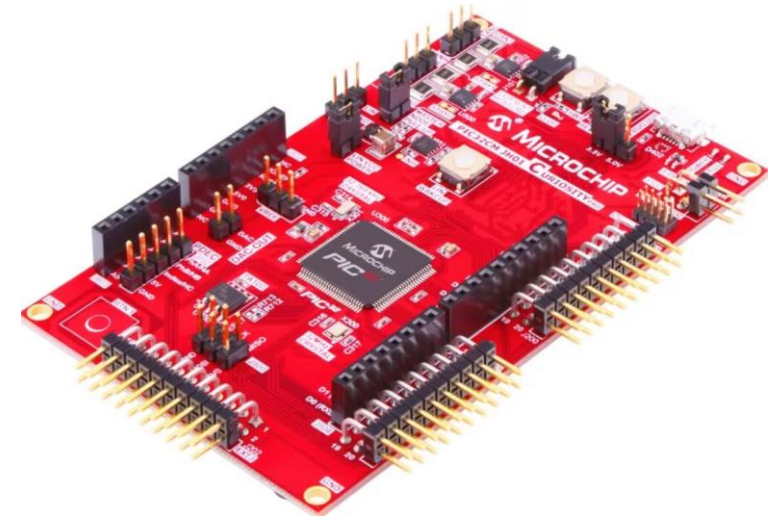
1. Copy the Startup code file pic32c/lib/proc/<device>/startup_<device>.c to the project dir
2. Customize the Startup code.
3. In MPLAB® X IDE select “Do not link device startup code”
4. Add the modified startup code to MPLAB® X IDE “Source Files” tab.



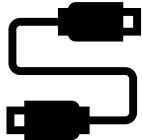
C Startup Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro board to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Show the references to the startup code, `_on_reset`, in `startup_xc32.c`**
- 3. Show the actual user customized startup code in function, `_on_reset`, located in the main c file.**
- 4. Run the program and see the resulting LED behavior of the customized startup code**

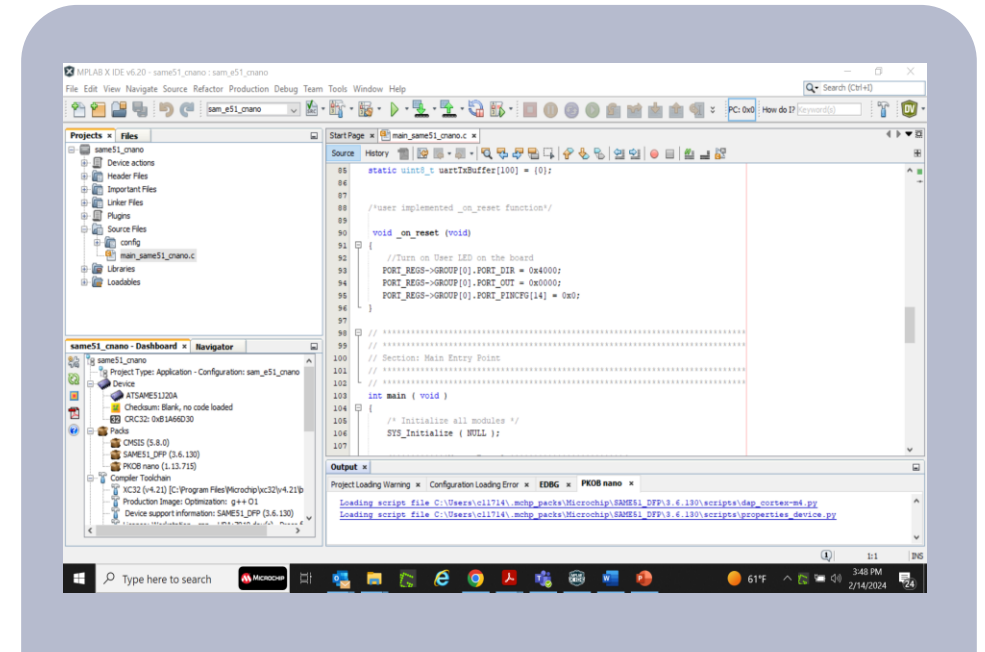
C Startup Demo



USB cable to PC(no
debugger required)



PIC32CM JH01 Curiosity Pro board



Agenda

- MPLAB[®] XC Specifics
 - C Startup
 - **Placing a variable/function at a specific address**
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
- Other Tools
 - Compiler Advisor
 - Analysis Tool Suite-Code Coverage
- MPLAB[®] X IDE Arm[®] Tools
 - printf/logging without using a UART
 - Live Connect
 - Trace back to Source

Placing a variable at a fixed memory address

- **Example: Bootloaders often need to have variables (like boot status and control) which are visible to the application and the boot loader. Unless you force these variables to a fixed address, the linker may move them when you recompile.**
- **Advisable to use volatile keyword to prevent optimization**
- **Place any absolute variables at beginning or end of memory to avoid memory fragmentation**
- **You cannot assign an absolute address to a local variable or function parameter**

Placing Variable at Fixed RAM Address

- **Declaration of variable**

```
int bootStatus @0x50;    // XC8
```

```
int bootStatus __attribute__((address(0x50))); //XC-DSC
```

```
int bootStatus __attribute__((address(0x50))); //XC32
```

```
int bootStatus __at(0x50); // With CCI syntax enabled
```

- **Verify address in Watch Window**

Output				Search Results				Watches ⌘			
Name				Type				Address			
☒ bootStatus				unsigned char				0x50			

Place a variable in Flash / ROM at fixed address (MPLAB® X XC8,XC-DSC,XC32 using CCI)

- Variable declaration

`const unsigned long myROMVariable __at(0x7FE) = 0x12345678;`

- Verify address in Program Memory

Output	Search Results		Watches	Program Memory %	
   	Line	Address	Opcode	Label	DisAssy
	2046	07FD	2802		GOTO 0x2
	2047	07FE	3478		RETLW 0x78
	2048	07FF	3456		RETLW 0x56
	2049	0800	3434		RETLW 0x34
	2050	0801	3412		RETLW 0x12
	2051	0802	3FFF		MOVWI -1 [FSR1]

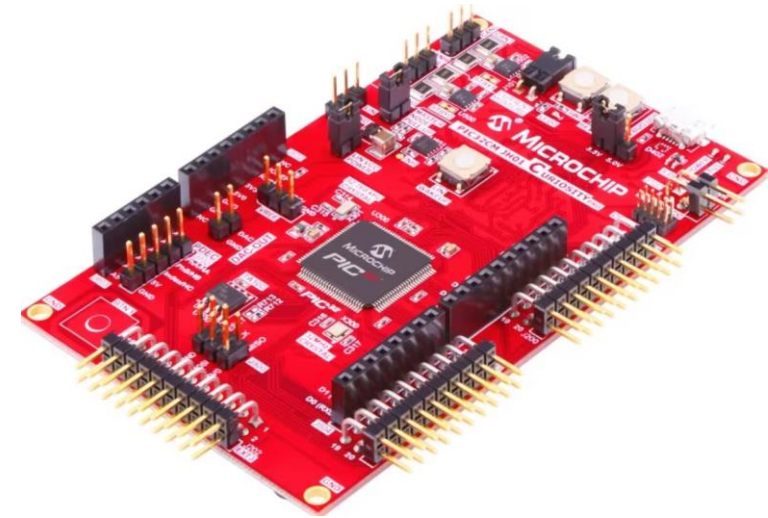
- Verify address in Watch Window

Output	Search Results	Watches			
	Name	Type	Address	Value	
	☒  bootStatus	unsigned char	 0x50		'x'; 0x78
	☒  myROMVariable	unsigned long	 0x7FE		0x12345678

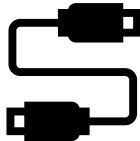
Variable @ Fixed Memory Location Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro board to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Show variable declaration, `mem_loc`, in the main C file**
- 3. Run debugger and halt**
- 4. Add `mem_loc` to Watch Window and note its address**

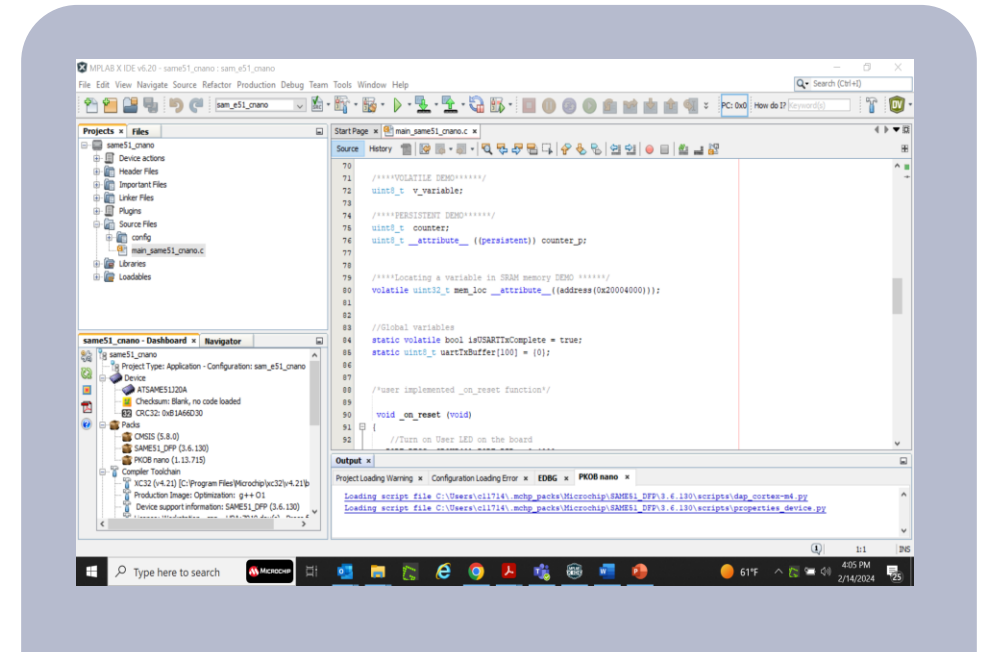
Variable @ Fixed Memory Location Demo



USB cable to PC(no
debugger required)



PIC32CM JH01 Curiosity Pro board



Agenda

- MPLAB[®] XC Specifics
 - C Startup
 - Placing a variable/function at a specific address
 - **Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack**
- Other Tools
 - Compiler Advisor
 - Analysis Tool Suite-Code Coverage
- New MPLAB[®] X IDE Arm[®] Tools
 - printf/logging without using a UART
 - Live Connect
 - Trace back to Source

MPLAB® X XC8 Hardware Stack for PIC MCU

- Default stack used for PC return addresses.
- Has a fixed limited size.
- Compiler tracks the Hardware stack usage for various functions during compile, emits warning for stack overflow. Cannot predict interrupt usage.
- **MPLAB® X XC8 Managed Stack option(non PIC18 devices)**
 - In MPLAB® X, Project Prop>XC8 Global Options>XC8 Linker>Runtime>Managed Stack
 - Enables alternate means of calling functions to allow deeper function nesting.
 - Compiler reverts to a look-up table method of calling functions using jumps once the HW stack is full. It does come at the expense of memory and program speed. Also, reentrancy is not allowed when using this option.

MPLAB® X XC8 PIC® MCU Compiled Stack

- Defaults to a *Compiled Stack* for function parameters, local variables
- Reentrancy is not allowed in MPLAB®X XC8 for PIC MCUs with the **Compiled Stack**, due to overlapping parameters
 - This includes Recursion and Indirect Recursion
- **“Buddy Functions”**
 - All functions called through a common *function pointer* all share the same parameter memory location
 - These functions should not call each other
- **Can optionally specify reentrant(Software stack) or hybrid stack(can specify reentrant/non-reentrant for each function) for Enhanced Midrange PIC® MCUs and PIC18 devices.**

MPLAB® X XC8 Stack Management

- **How is the default stack sized?**

- Largest contiguous block of data RAM available
- For MPLAB® X XC8-AVR, it is assigned at the highest RAM location and built down.

No hw stack, as the sw stack is used for both function return addresses and stack based objects allocated by functions.

- **How to determine the required size?**

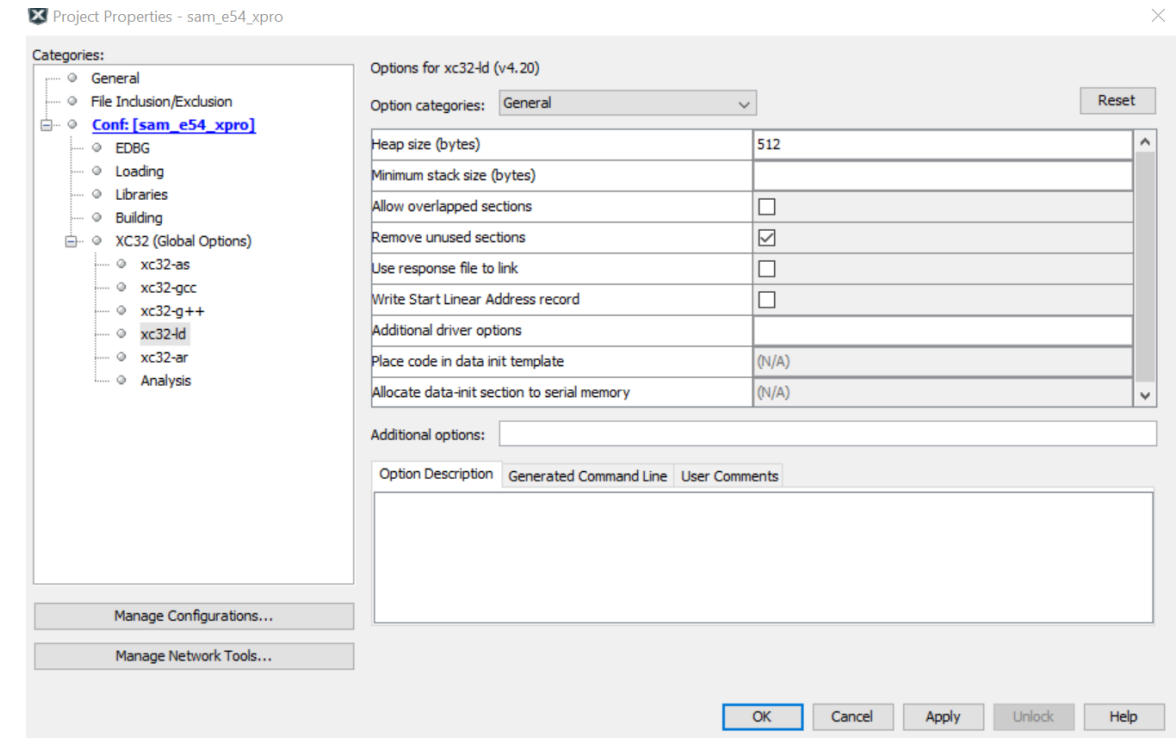
- Calls + parameters + locals
- On deepest call depth + ISRs
- Call graph helps (list file, IDE call graph)

- **Set maximum sw stack size in MPLAB® X IDE project properties**

- Only for Enhanced Midrange PIC® and PIC18® devices

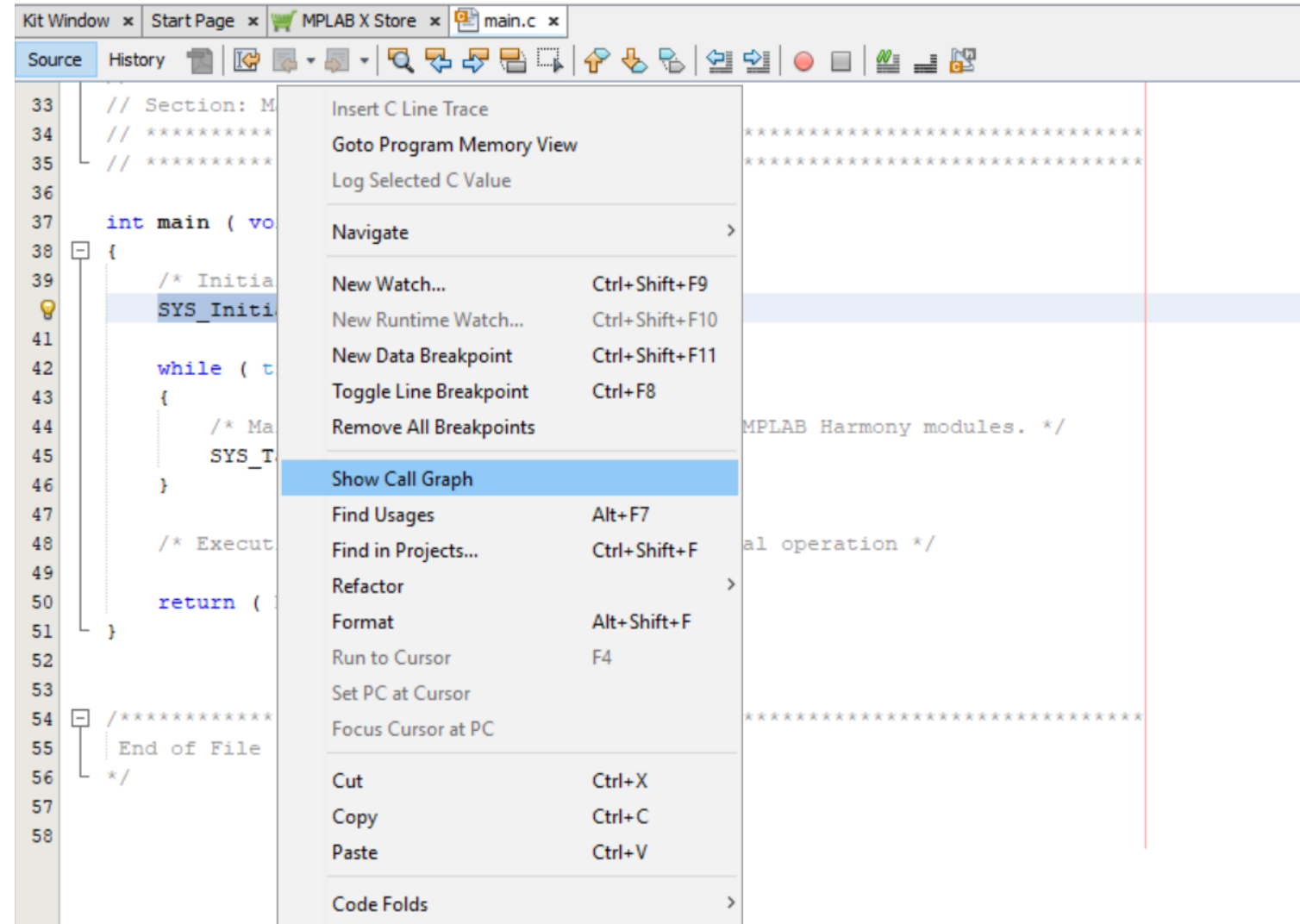
MPLAB® X XC-DSC, XC32 Stack and Heap

- MPLAB® X XC-DSC automatically allocate the stack & heap to the largest contiguous block
- No need to specify the stack/heap in the linker script
- No overflow flag in MPLAB® X XC32, can set SPLIM in XC-DSC
- Specify the minimum stack & heap sizes in the project properties(MPLAB® X XC-DSC default is 16 bytes, XC32 is 1024 bytes)

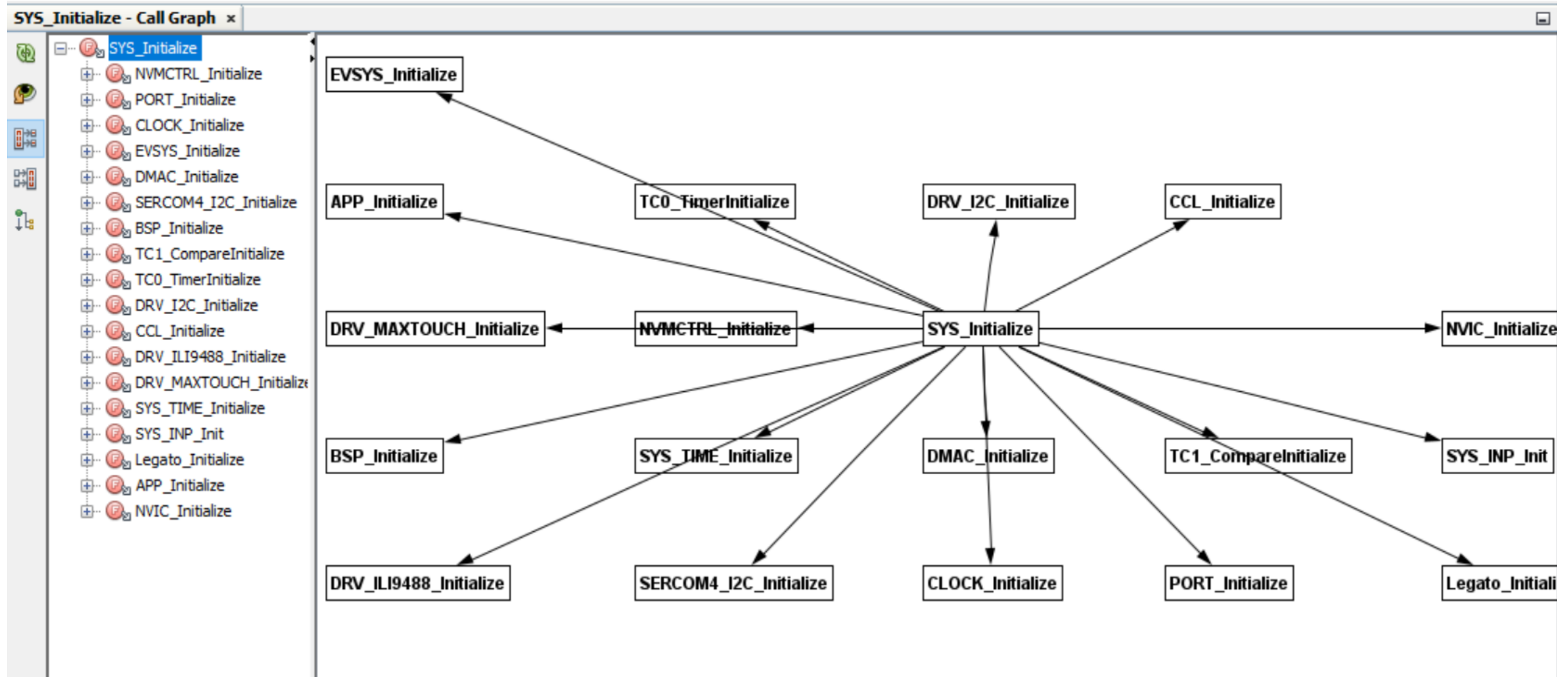


How to Display the Call Graph

Right Click on Function,
“Show Call Graph”



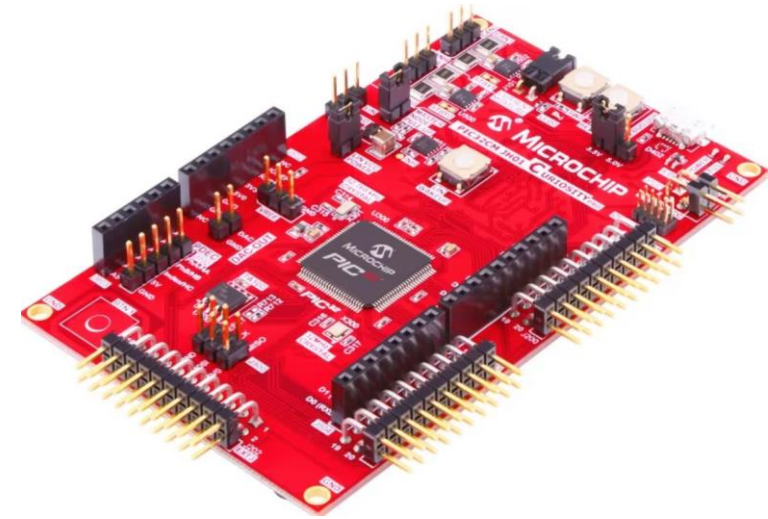
Call Graph



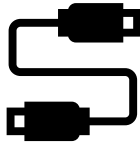
Call Graph Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro board to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Build and clean project**
- 3. Right click on the main function, and show call graph**
- 4. Go over the various different views of the Call Graph**

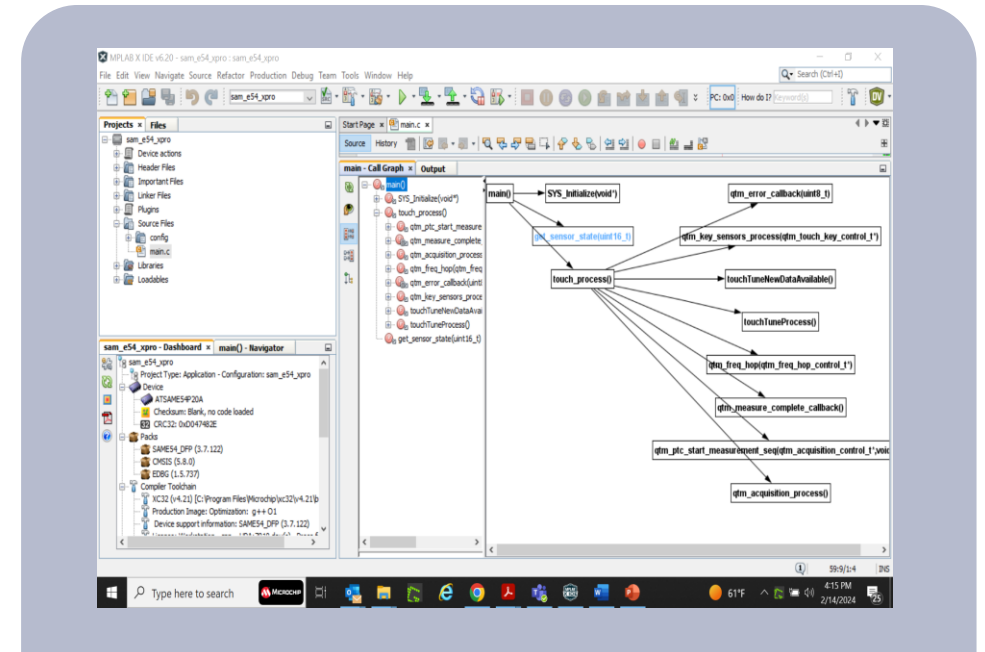
Call Graph Demo



USB cable to PC(no
debugger required)

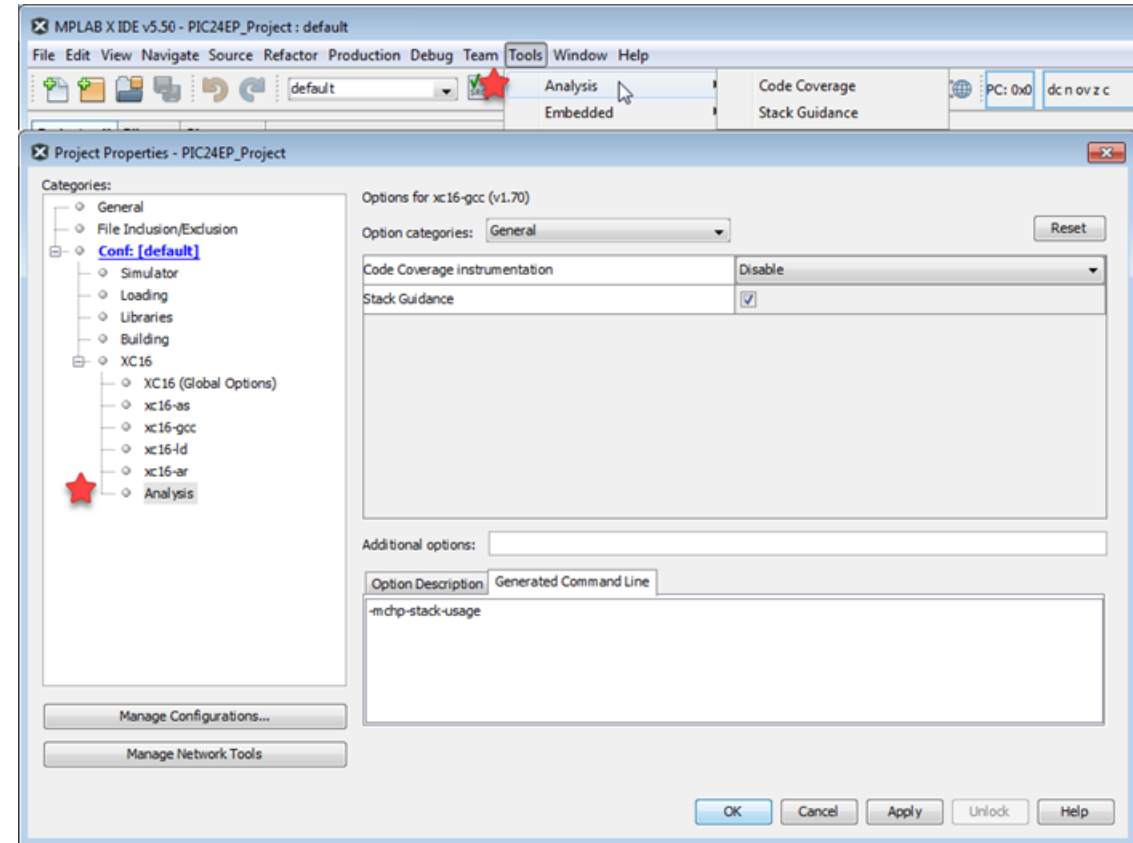


PIC32CM JH01 Curiosity Pro board



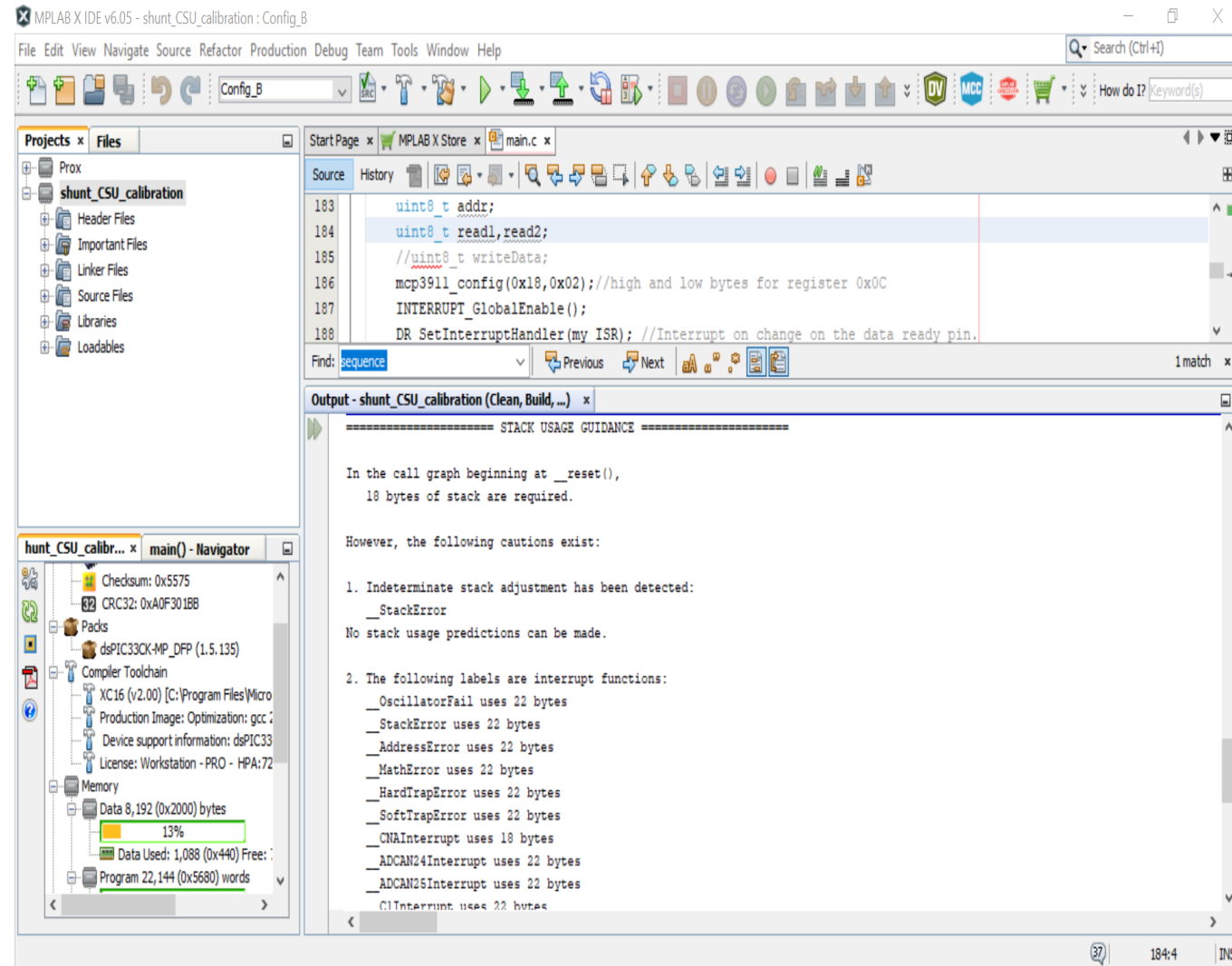
Stack Guidance

- Performs a compile time static analysis
- Guides you to an appropriate worst-case stack usage requirement
- Information is provided in Output window and .map file
- Available with MPLAB® X XC8, XC-DSC, XC32 PRO licenses



Stack Guidance(cont.)

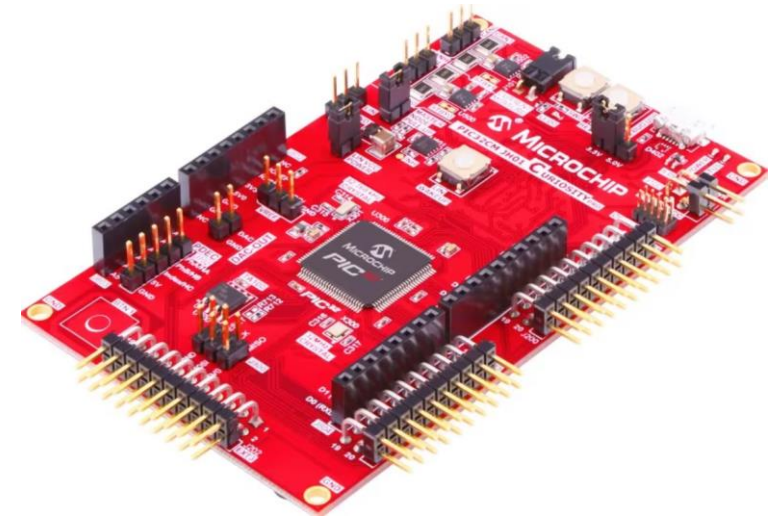
- Estimates usage of hw and sw stacks
- Since it is a static analysis, it cannot see dynamic use of the stack
- It cannot accurately estimate stack usage with the following:
 - recursion
 - interrupts,
 - functions called by pointers
 - Inline assembly
 - debugger use of hw stack



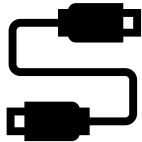
Stack Guidance Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Enable Stack Guidance in the Tools menu**
- 3. Build project**
- 4. Go over the Stack Guidance output in the Output window**

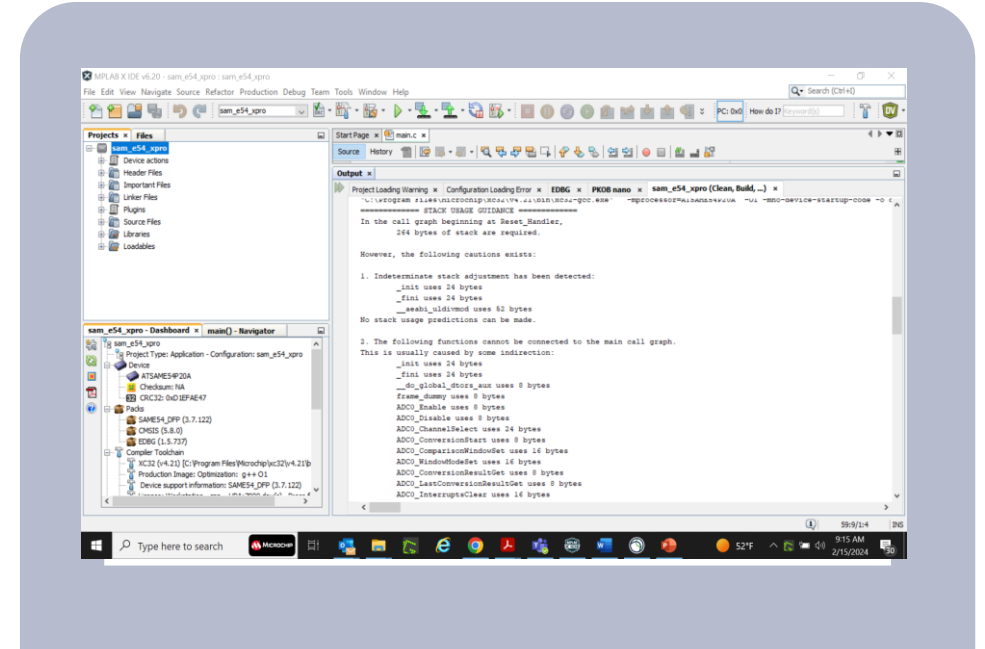
Stack Guidance Demo



USB cable to PC(no
debugger required)

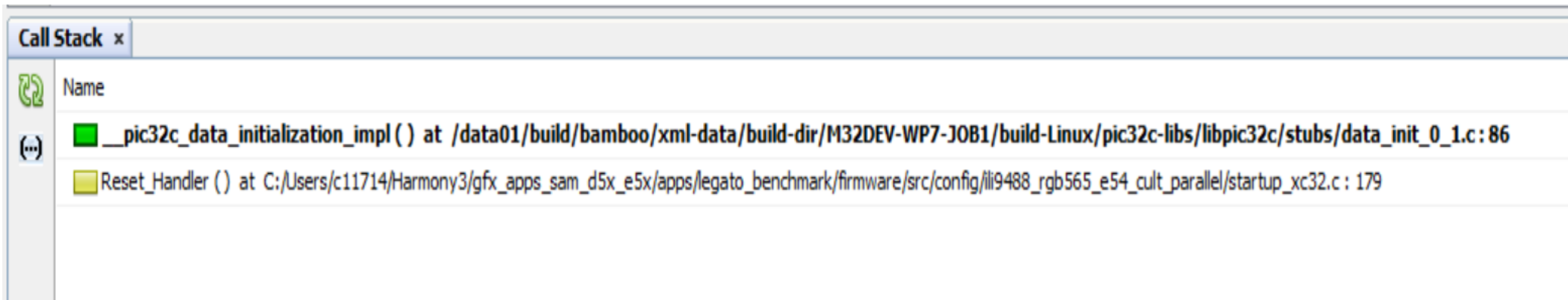


PIC32CM JH01 Curiosity Pro board



Call Stack

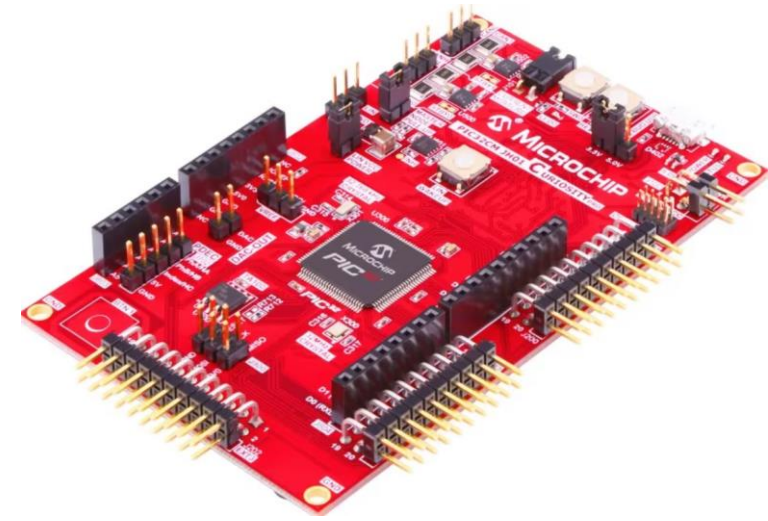
- Window>Debugging>Call Stack
- Displays C functions which are currently part of the stack and their arguments listed in the order in which they were called
- Available for 16- and 32-bit devices.
- The window is updated on Debug Pause or Halt.
- Will not be accurate if halted within a library function or inline function



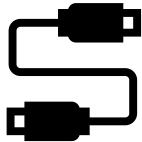
Call Stack Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Run debugger and halt**
- 3. Open call stack Window in the Window menu**
- 4. Run/halt debugger several times to see the updates in the Call Stack window**

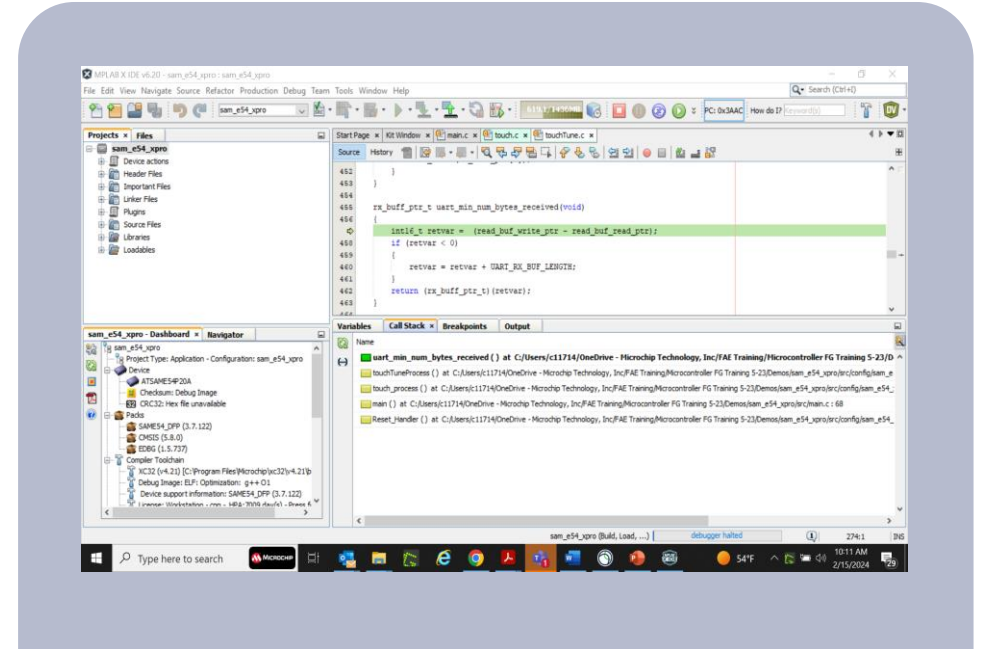
Call Stack Demo



USB cable to PC(no
debugger required)



PIC32CM JH01 Curiosity Pro board

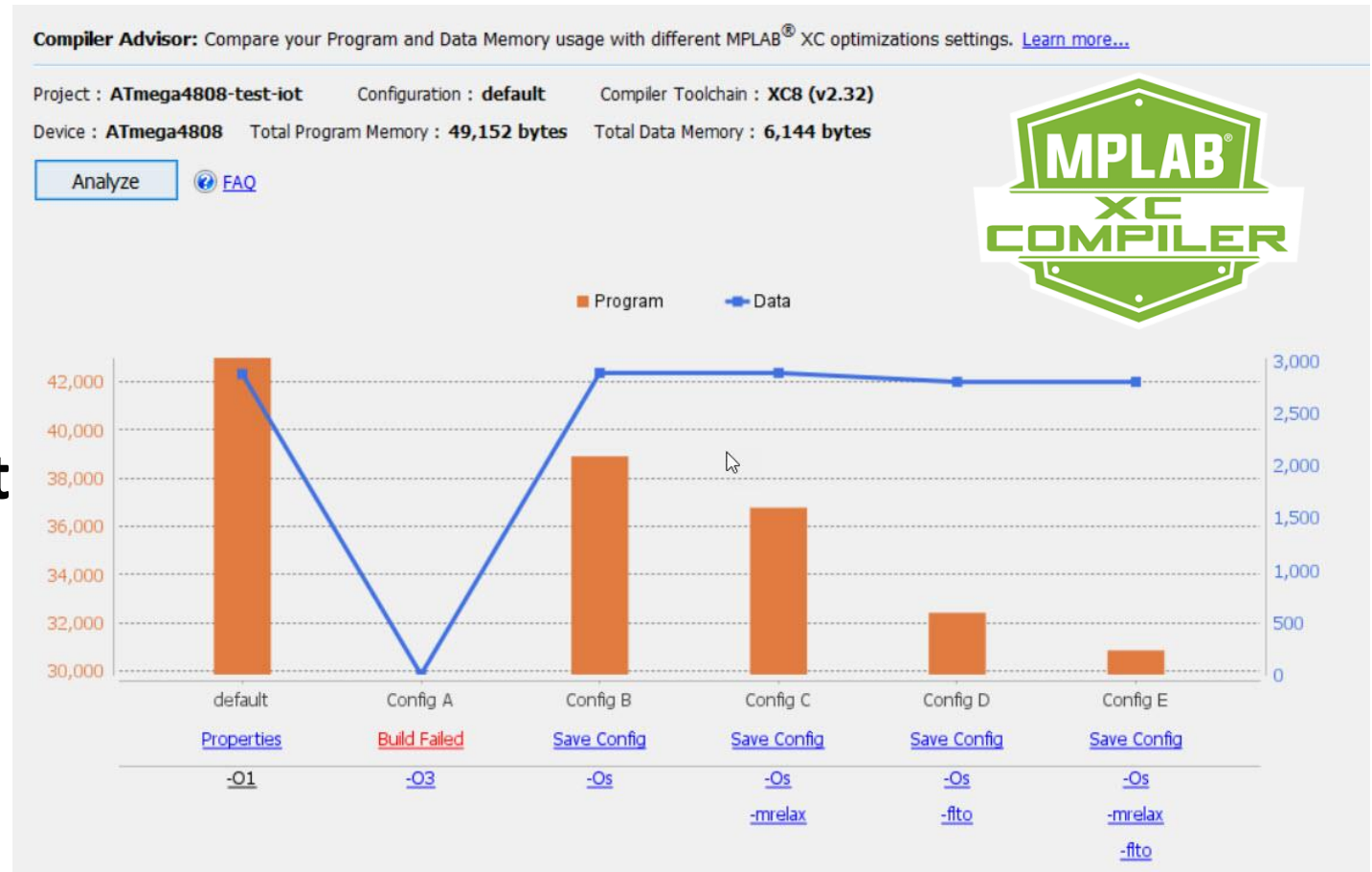


Agenda

- MPLAB[®] XC Specifics
 - C Startup
 - Placing a variable/function at a specific address
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
- Other Tools
 - **Compiler Advisor**
 - Analysis Tool Suite-Code Coverage
- MPLAB[®] X IDE Arm[®] Tools
 - printf/logging without using a UART
 - Live Connect
 - Trace back to Source

Compiler Advisor

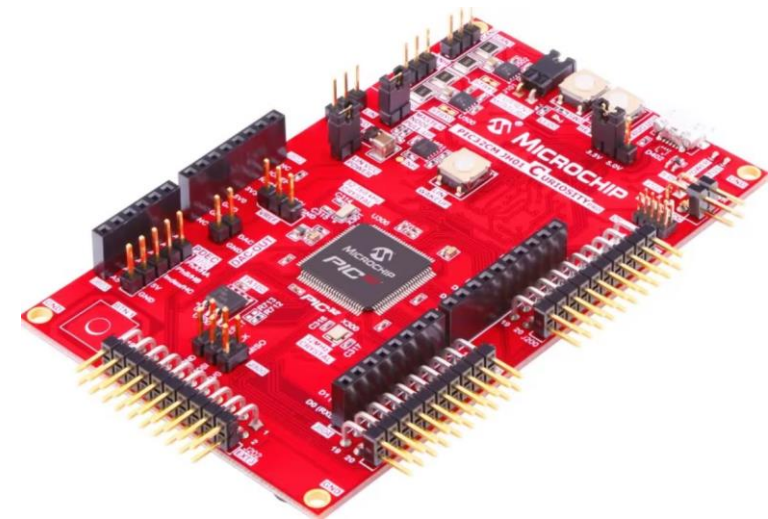
- See how each optimization affects your code size
 - Get detailed information on what each optimization does
- Choose the best optimizations for your project and device
- Supports all compiler types
MPLAB[®]X XC8, XC-DSC, XC32
- Available with free or PRO licenses



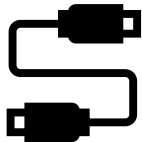
Compiler Advisor Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Enable Compiler Advisor in the Tools menu**
- 3. Push Analyze in the Compiler Advisor window**
- 4. Explain the different options shown in the window**

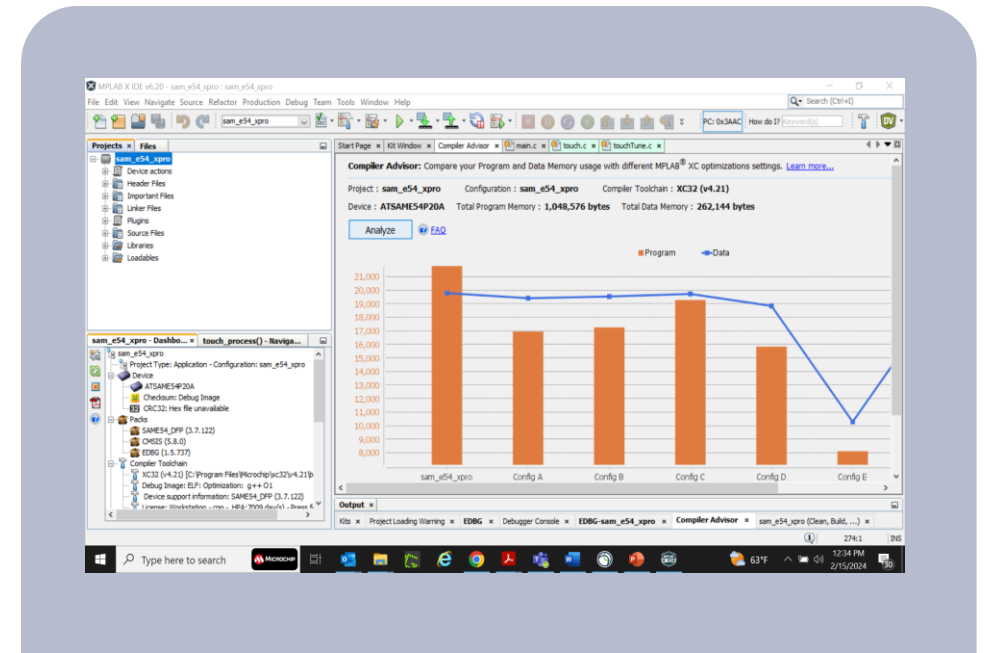
Compiler Advisor Demo



USB cable to PC(no
debugger required)



PIC32CM JH01 Curiosity Pro board



Agenda

- MPLAB[®] XC Specifics
 - C Startup
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
 - Placing a variable/function at a specific address
- Tools
 - Compiler Advisor
 - Analysis Tool Suite- Code Coverage
- MPLAB[®] X IDE Arm[®] Tools
 - printf/logging without using a UART
 - Live Connect
 - Trace back to Source

MPLAB[®] Analysis Tool Suite

**Evaluate Your Code Coverage
Without the Slowdown**

Now with MISRA Rules Checking

LEARN MORE >>



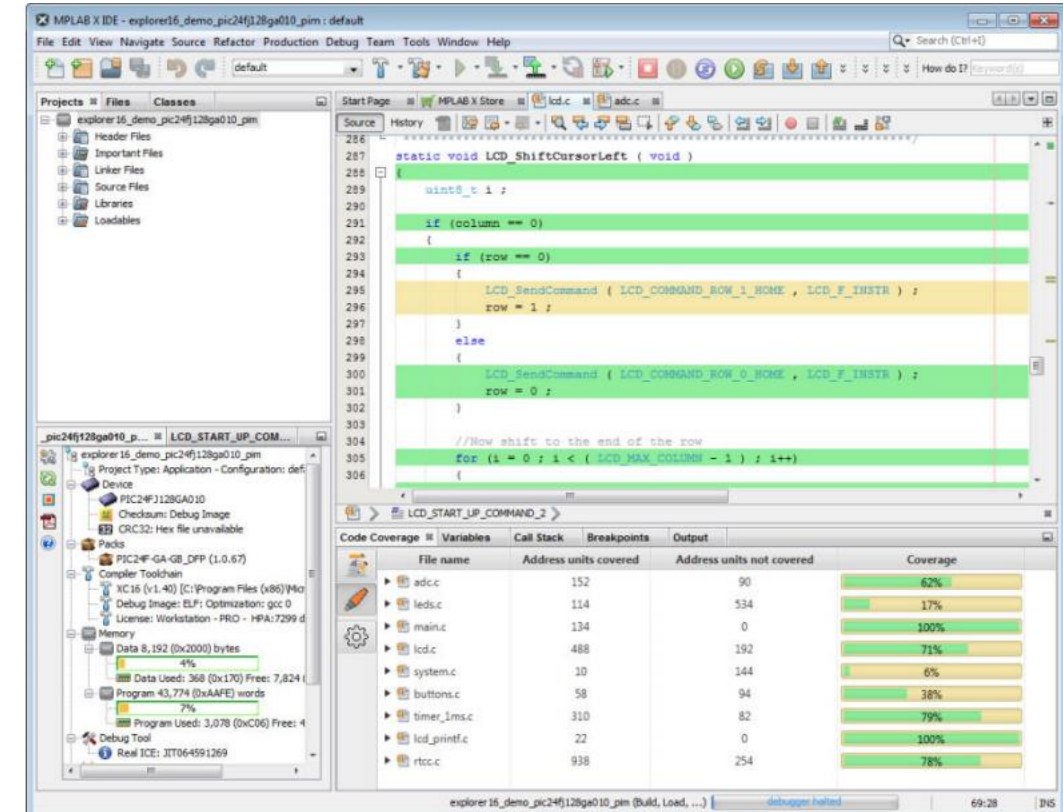
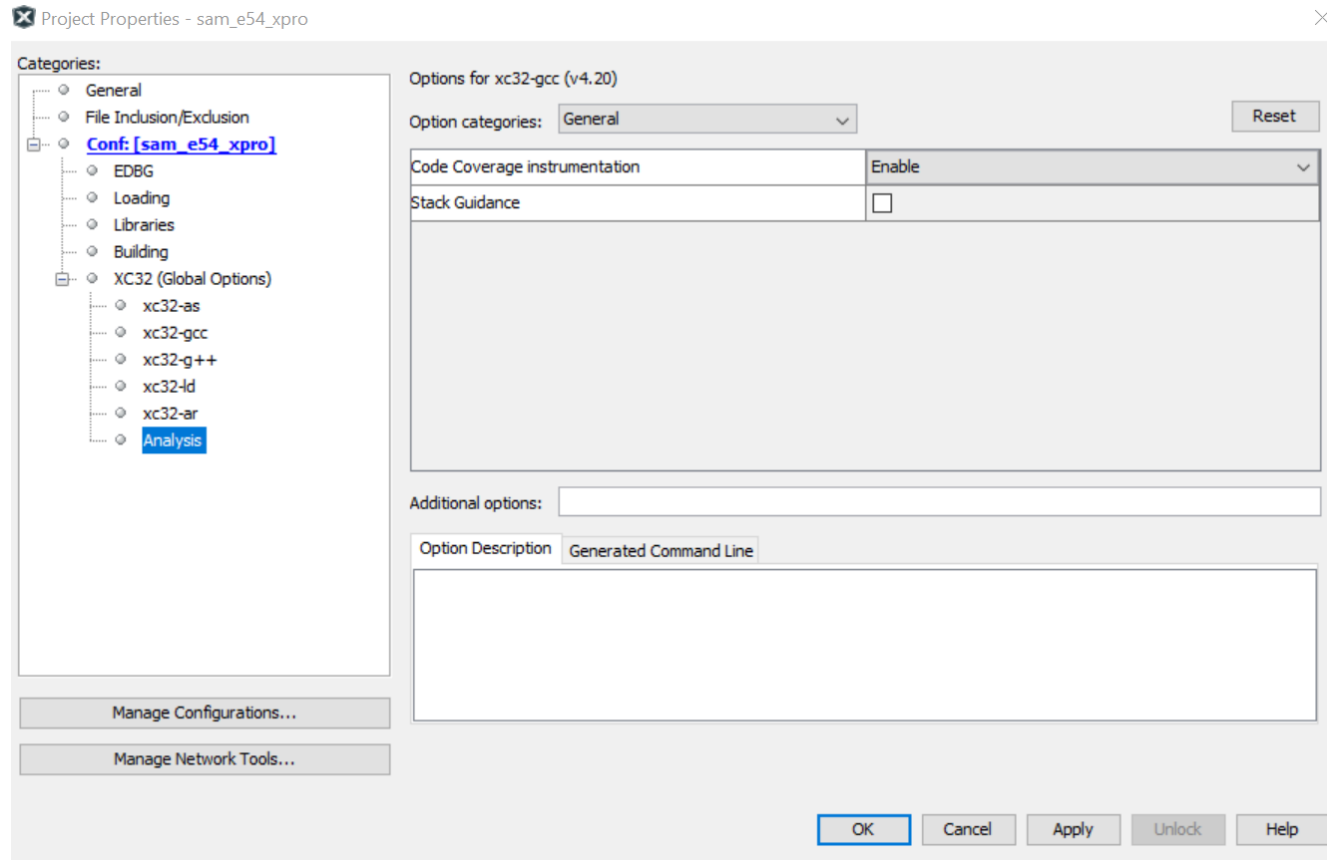
MPLAB[®] Analysis Tools Suite

- **Code Coverage and MISRA Rules checking in one product**
 - Be compliant with MISRA coding standards
 - 140 rules checked
 - Now includes Thread Safety
 - Easily see which code has been executed
 - Run tests untethered
 - Minimal impact to program memory and execution speed
 - Typical addition to code size is <1%
 - Create custom reports and configurable summary views
 - Works with any MPLAB[®] XC C Compiler(min versions: XC8-v2.35v,XC16 v2.0, XC32 v4.0 or later)
 - Requires MPLAB[®] X v6.0 or later



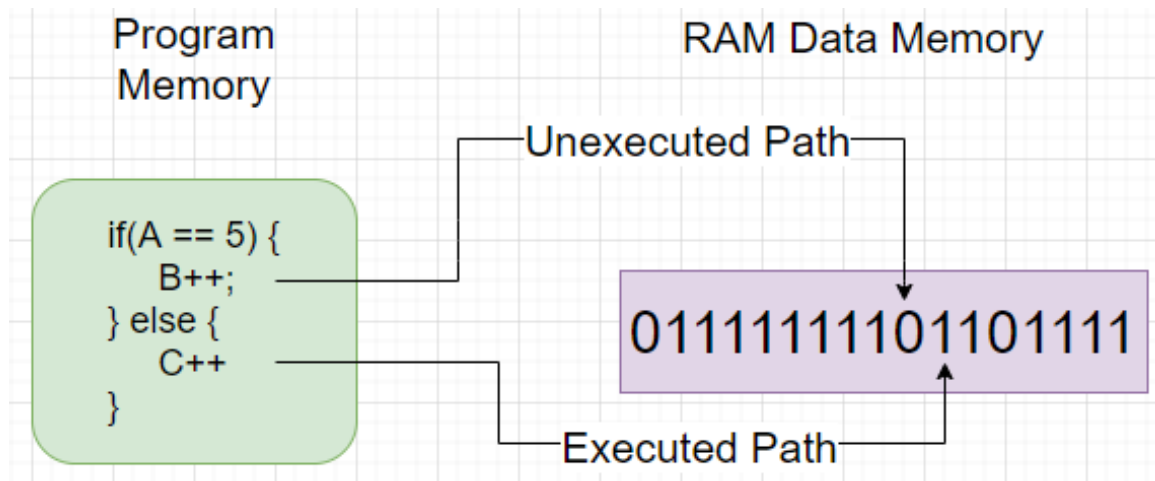
MPLAB® Code Coverage

Display which lines of code have executed and which have not



<https://vimeo.com/358081157>

MPLAB[®] Code Coverage

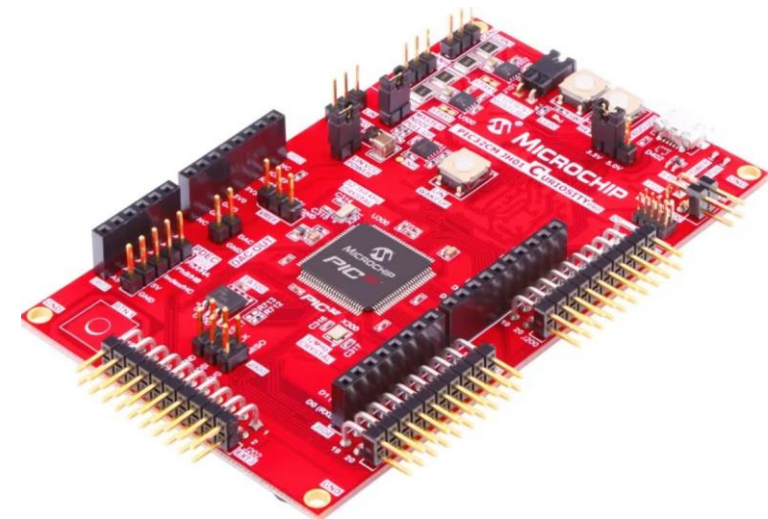


- Code Coverage is supported on any debug tool supported by MPLAB[®] X IDE.
- Works with free or PRO version of XC compilers
- Compiler adds a minimal amount of instrumented assembly code to update flags in RAM to indicate coverage. To remove this code from the application when development ends, disable code coverage
- Adds highlighted colors to code: green = executed, yellow = partially executed, and red = not executed to editor and program memory windows
- Code coverage is ELF only

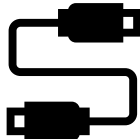
Code Coverage Demo

- 1. We will connect a PIC32CM JH01 Curiosity Pro to the PC and open the Harmony touch example project `pic32cmjh_curiosity_pro.x`**
- 2. Enable Code Coverage in the Tools menu**
- 3. Open Code Coverage Window in the Window menu**
- 4. Run debugger and halt**
- 5. Explain the different options shown in the window**

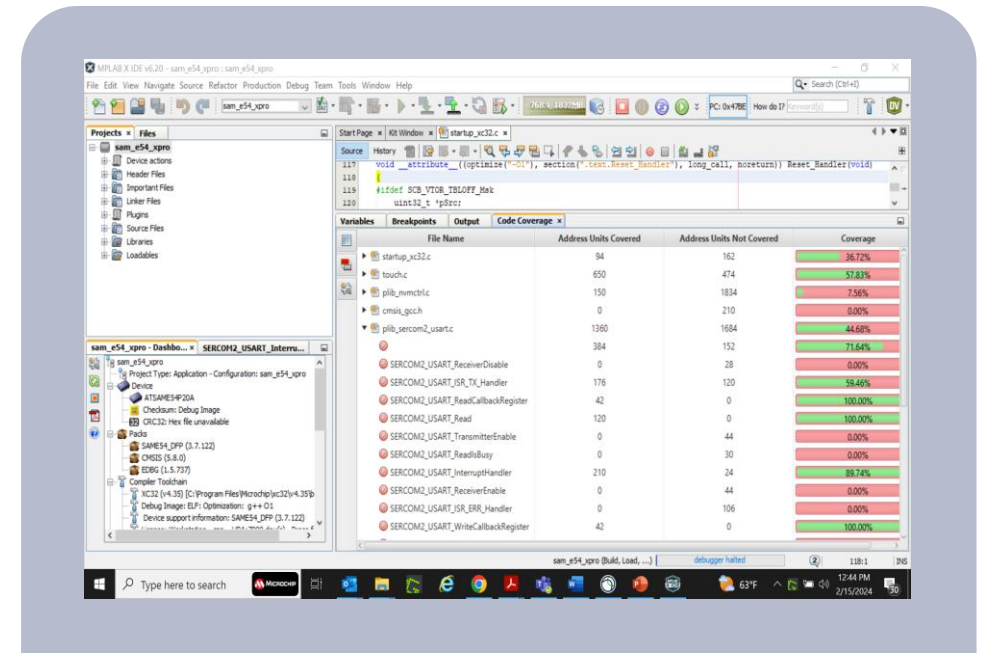
Code Coverage Demo



USB cable to PC(no
debugger required)



PIC32CM JH01 Curiosity Pro board



Agenda

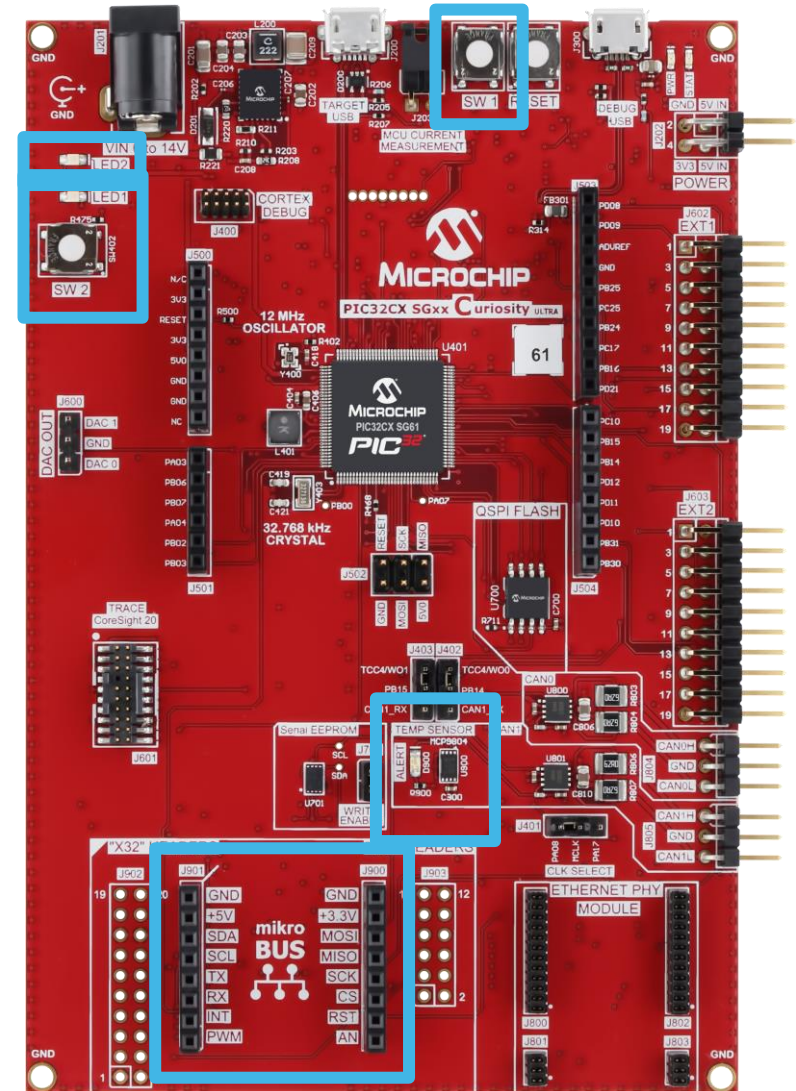
- **MPLAB[®] XC Specifics**
 - C Startup
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
 - Placing a variable/function at a specific address
- **Tools**
 - Compiler Advisor
 - Analysis Tool Suite- Code Coverage
- **MPLAB[®] X IDE Arm[®] Tools**
 - **printf/logging without using a UART**
 - Live Connect
 - Trace back to Source



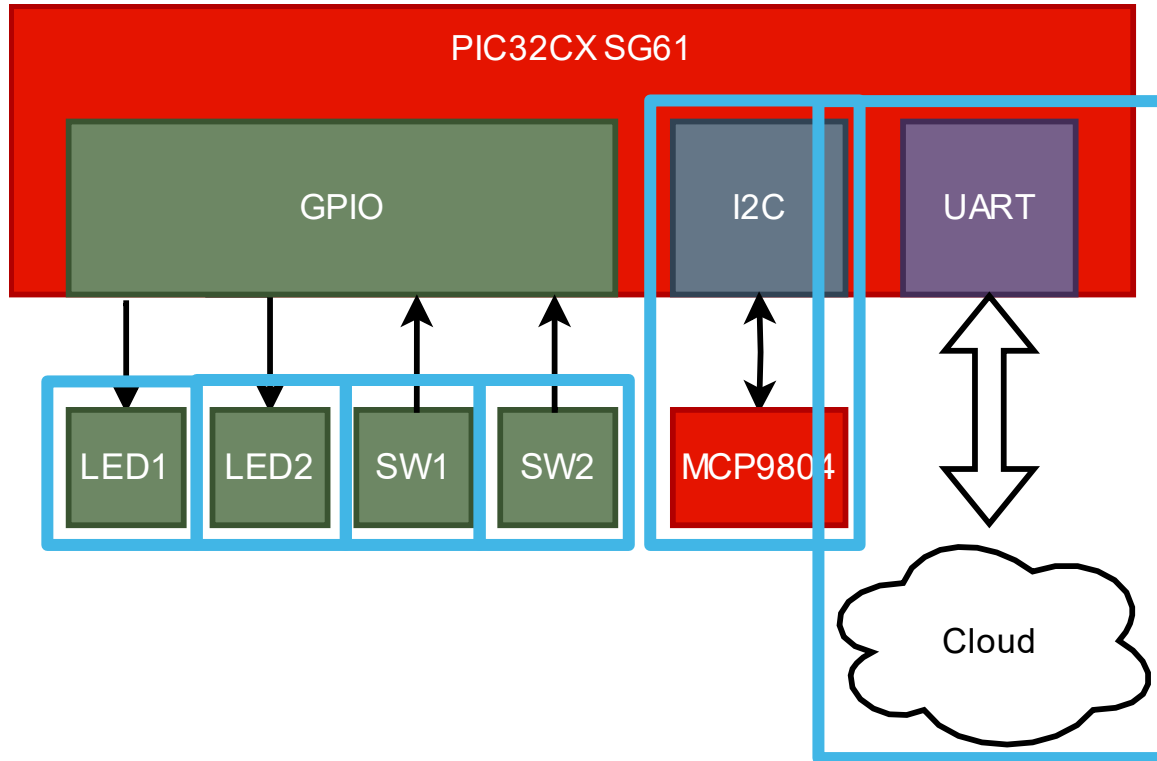
Adventures in Debugging

Application Description

- LED2 Blinks at 500mS interval
- Switch 2 illuminates LED1 when pressed
- Switch 1 prints current temperature
- UART reports activity of switches and temperature



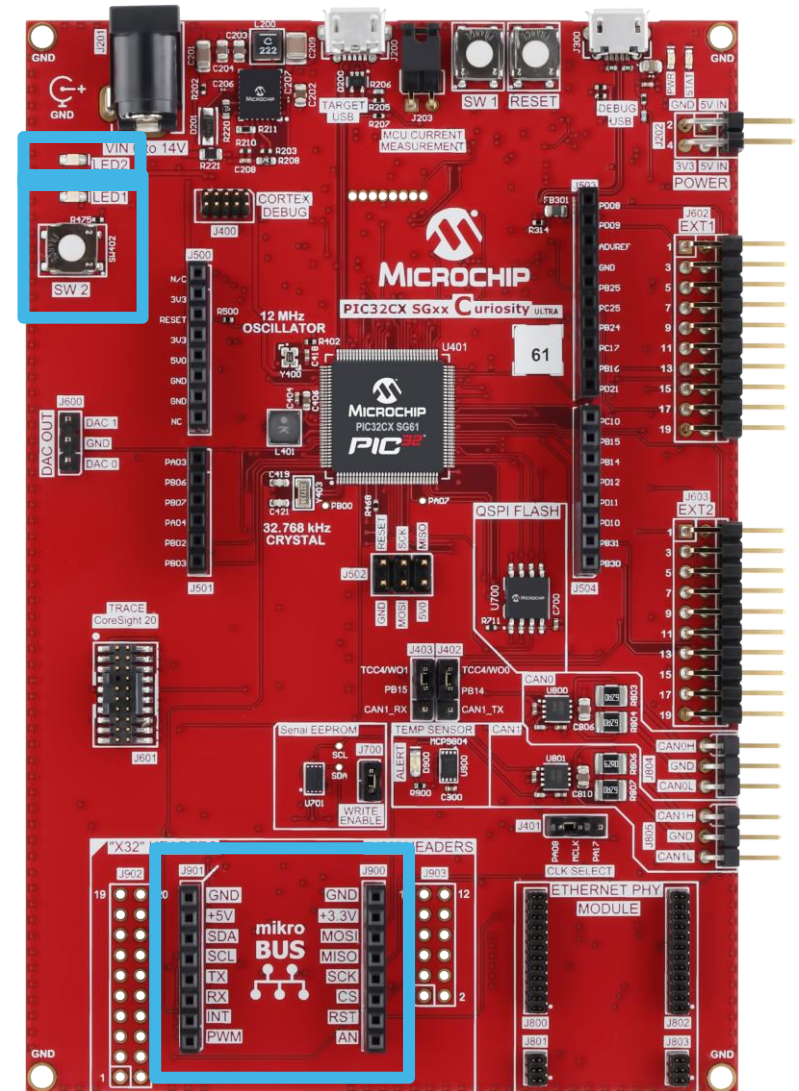
System View of Application



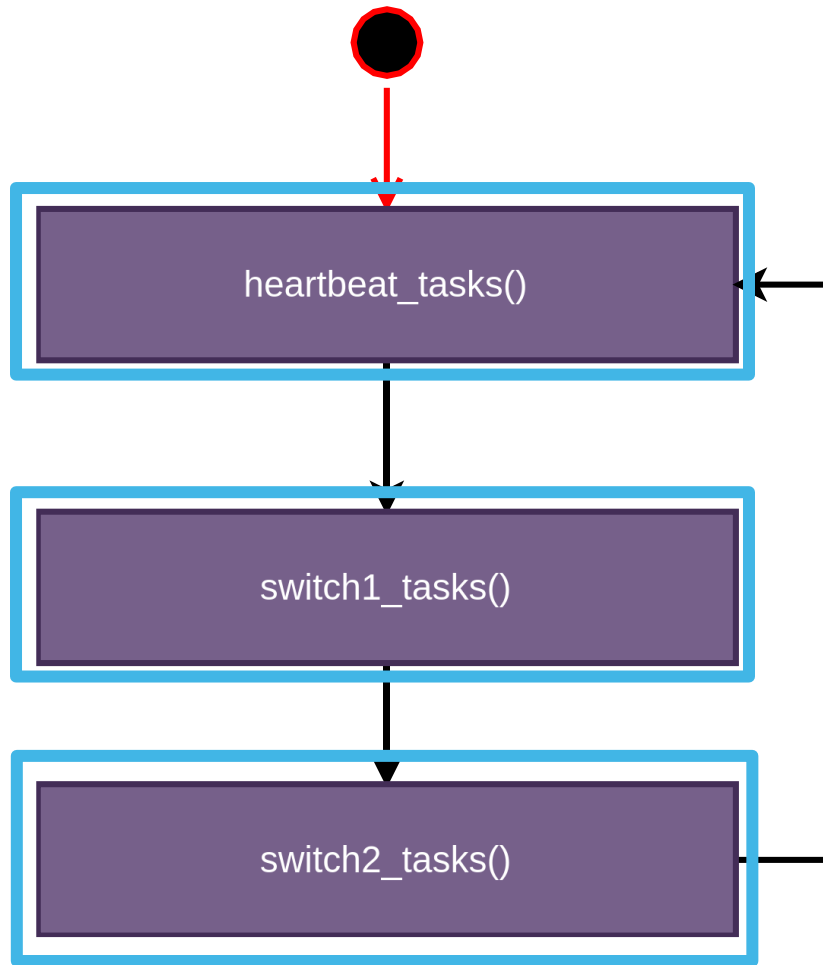
- **SW1** represents an async signal
 - Response is to measure temperature of a process
- **MCP9804** measures temperature
 - Communicates via I2C
- **SW2** represents an async user input
 - UI control to change the "display" to an alternate view
- **LED1** shows alternate view
 - Turns on when SW2 is held
- **LED2** shows system health
 - Timer driven heartbeat
- **UART** represents communication channel
 - Cannot interleave with debug information

First Run

- Project runs
- LED2 Blinks
- SW2 doesn't work as expected
- No LED1 and no UART output



State Diagram

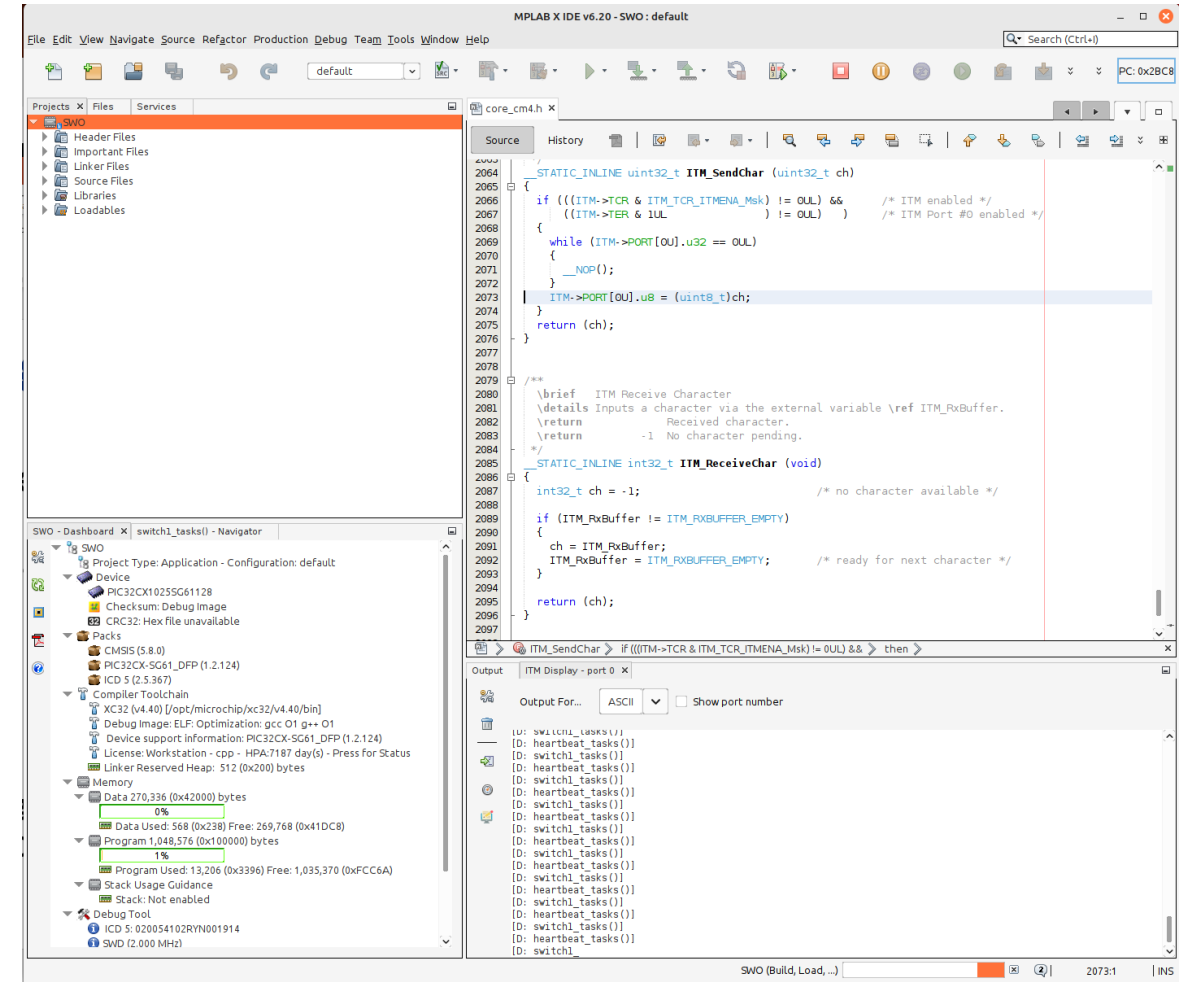


System States

- `heartbeat_tasks()`
- `switch1_tasks()`
- `switch2_tasks()`

Debug Logging

- Common tool used for embedded developers
- Output used to view internal state
- Methods
 - UART
 - Semi-Hosting
 - Single Wire Output (SWO)



UART

Benefits

- Easy to implement
- Done it before
- May already be in-system

Concerns

- Impacts application timing
- Consumes I/O pins
- Slow serial data rate

Semi-Hosting

Benefits

- **Uses debug interface**
- **Bi-directional with host**
- **No additional I/O consumed**

Concerns

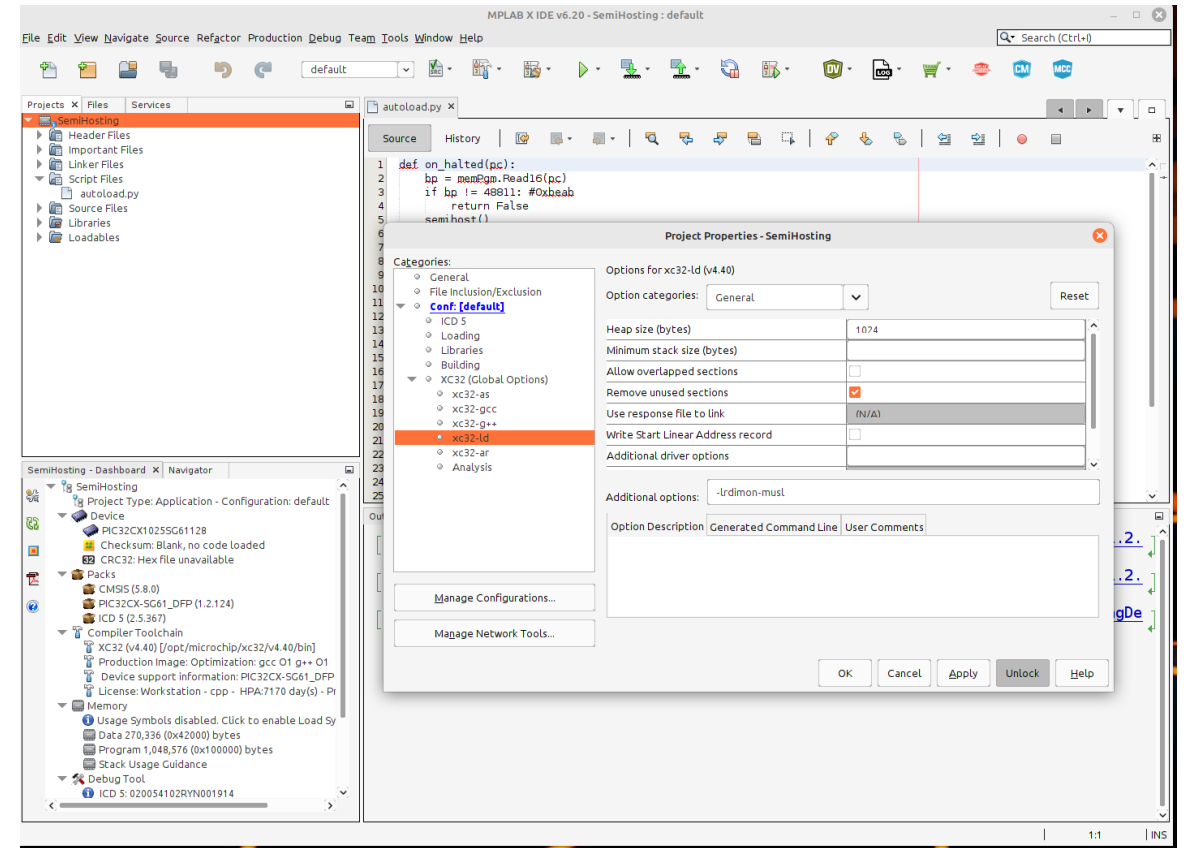
- **Requires debug session**
- **Impacts timing severely**
- **More difficult to implement**

Semi-Hosting Demo

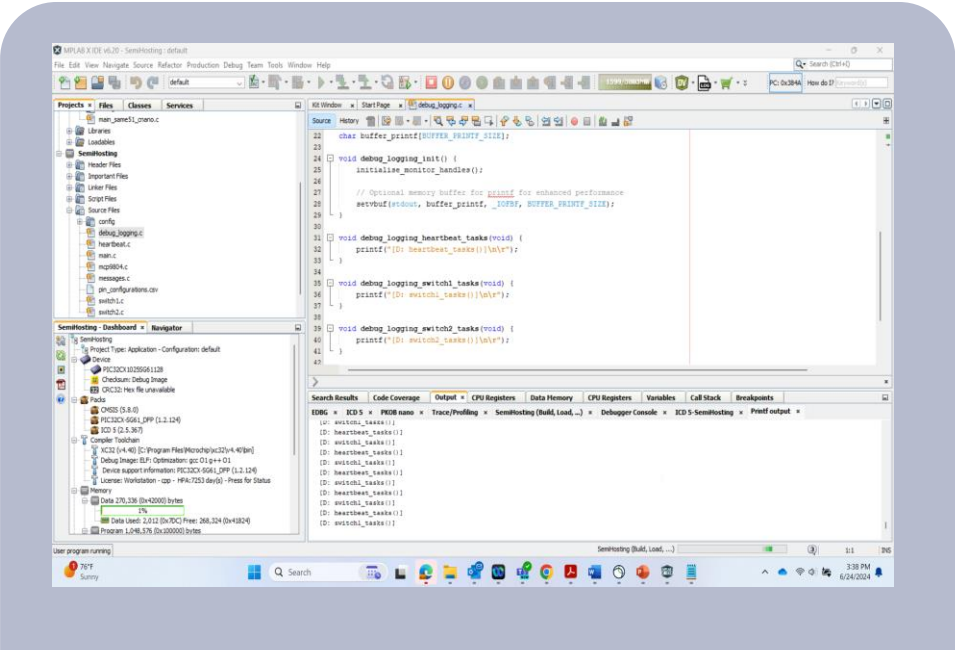
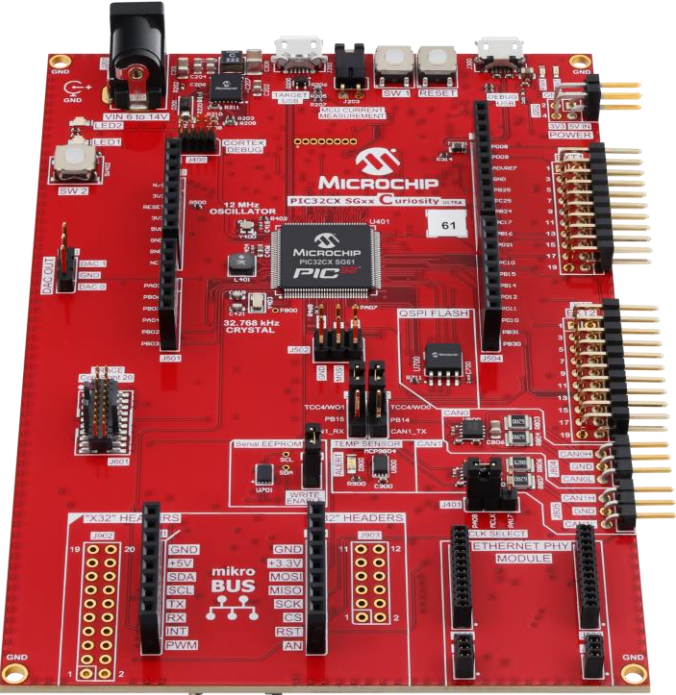
- 1. We will connect a PIC32CX SG61 Curiosity Ultra and an ICD5 to the PC and open the semi hosting example project**
- 2. Enable semi hosting via the Project Properties, source code addition, and adding a python script to the project**
- 3. Demonstrate the use of printf in the project via the output window.**

Semi-Hosting Setup

- Steps for enabling Semi-Hosting on a project
 - Add library in xc32-ld
 - Increase heap to 1k
 - Add autoload.py and copy contents from documentation
 - Add extern and call to initialize_monitor_handles()
 - Use printf for messages to appear in printf window
 - printf Window is generated by autoload.py
- [Documentation Link](#)



Semi-Hosting Demo



PIC32CX SG61 Curiosity Ultra board



Single Wire Output (SWO)

Benefits

- **Uses debug interface**
- **Fast serial data rates**
- **Always available**

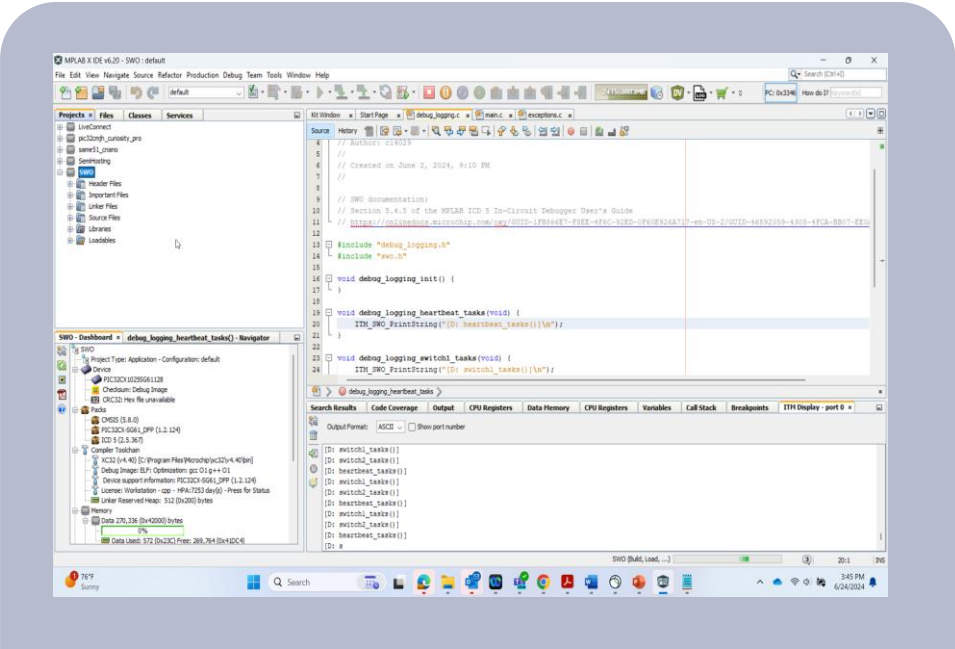
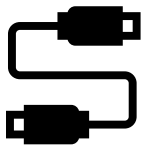
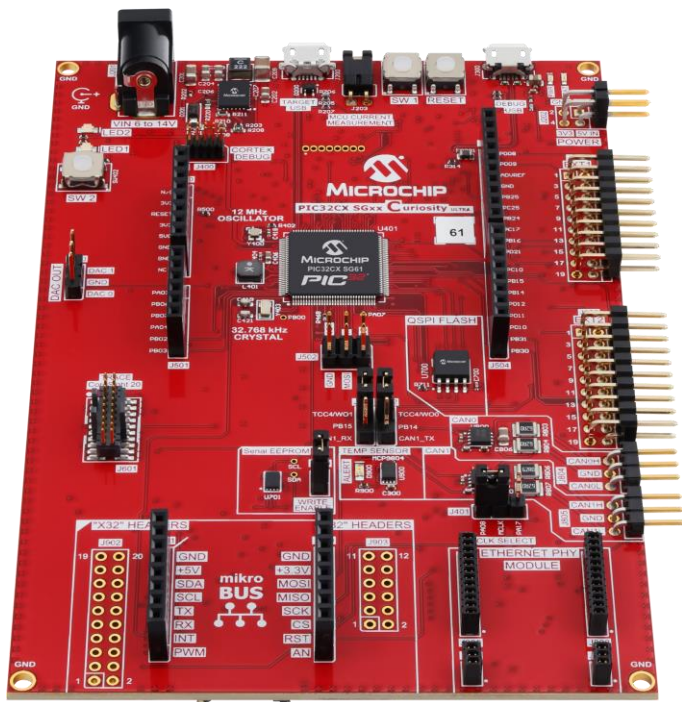
Concerns

- **Not available on all cores**
- **MPLAB X limitations**

SWO Demo

- 1. We will connect a PIC32CX SG61 Curiosity Ultra and an ICD5 to the PC and open the SWO example project**
- 2. Enable SWO via the Project Properties and source code addition.**
- 3. Demonstrate the use of ITO_SWO_PrintString in the project via the output window.**

SWO Demo

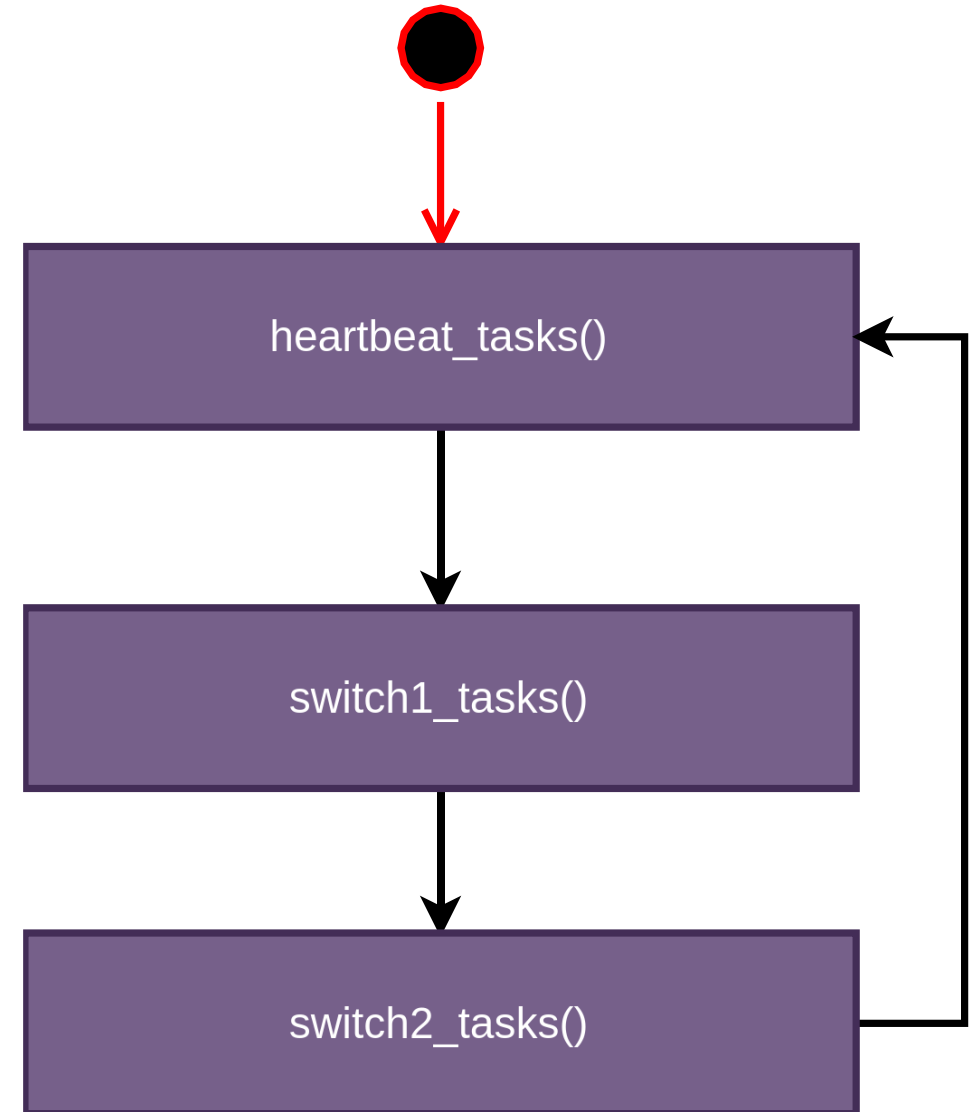


PIC32CX SG61 Curiosity Ultra board

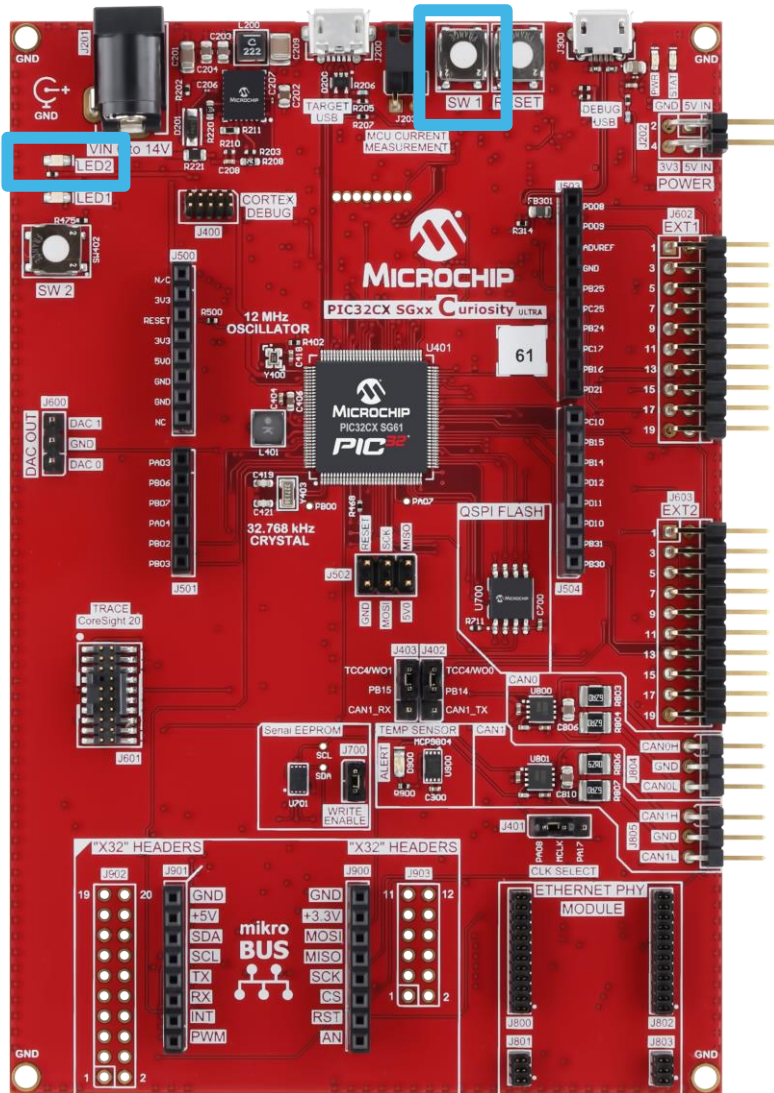


Second Run

- Enabled Debug Logging via SWO
- Does the output match the state diagram?



Third Run



- Debug Logging via SWO still enabled
- Testing SW1
- Heartbeat LED stops beating
- Why?
- What debug technique to use?

Agenda

- **MPLAB[®] XC Specifics**
 - C Startup
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
 - Placing a variable/function at a specific address
- **Tools**
 - Compiler Advisor
 - Analysis Tool Suite- Code Coverage
- **MPLAB[®] X IDE Arm[®] Tools**
 - printf/logging without using a UART
 - **Live Connect**
 - Trace back to Source

Live Connect Debug

Debug

The code will be built for debug, the device will be reset and reprogrammed by the hardware tool and a debug session will begin.

VS.

Live Connect Debug



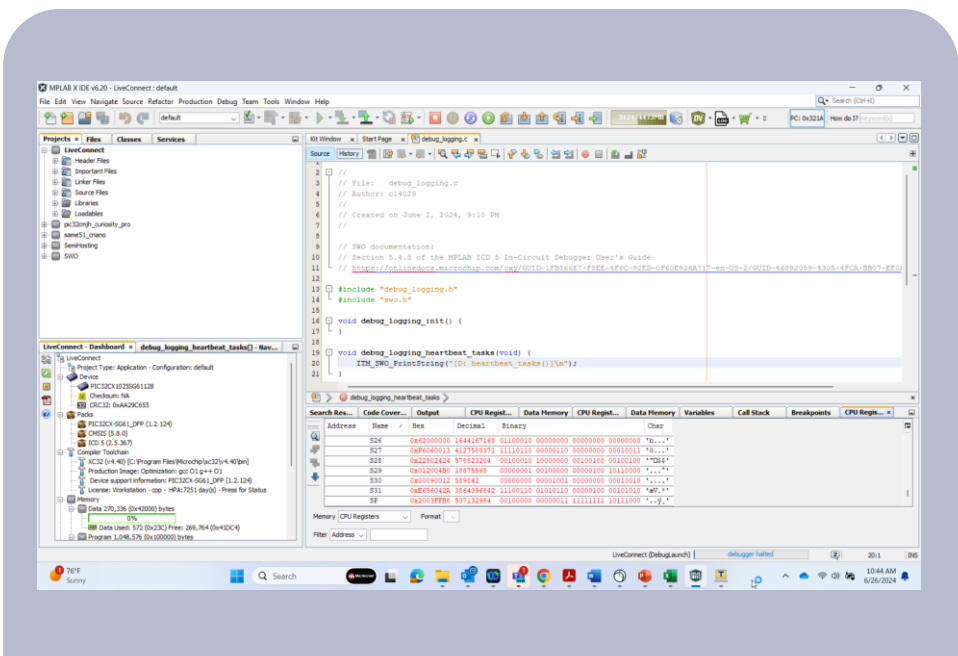
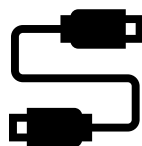
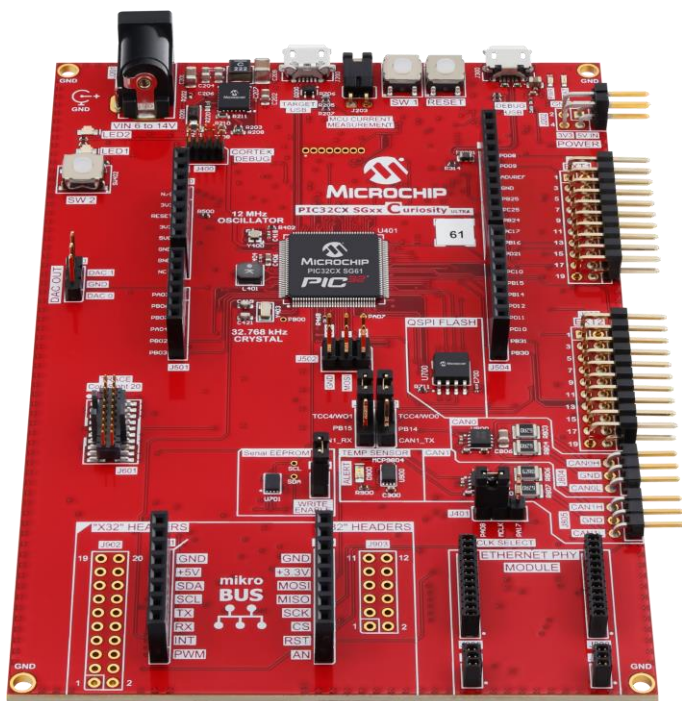
The hardware tool will connect, a debug session will begin, and the target will halt.

- **Before you program the device to run, enable Load Symbols to be able to debug with Live Connect Debug.**
- **Ensure the device is running before selecting Live Connect Debug. If the device is in reset it will halt in the reset vector handler and not in code.**
- **It is recommended that the reset pin is disconnected from the tool or target board to eliminate any spurious resets caused by any noise on the reset line.**

Live Connect Debug Demo

- 1. We will connect a PIC32CX SG61 Curiosity Ultra and an ICD5 to the PC and program the application into the board**
- 2. We will press SW1 to demonstrate the undesirable behavior**
- 3. We will show the difference between troubleshooting when source of fault is in library vs. source code**
- 4. We will demonstrate the use of Live Connect to gain visibility into the system during its current run**

Live Connect Debug Demo



PIC32CX SG61 Curiosity Ultra board



Agenda

- **MPLAB[®] XC Specifics**
 - C Startup
 - Stack Management-Tools used: Call Graph, Stack Guidance, Call Stack
 - Placing a variable/function at a specific address
- **Tools**
 - Compiler Advisor
 - Analysis Tool Suite- Code Coverage
- **MPLAB[®] X IDE Arm[®] Tools**
 - printf/logging without using a UART
 - Live Connect
 - **Trace back to Source**

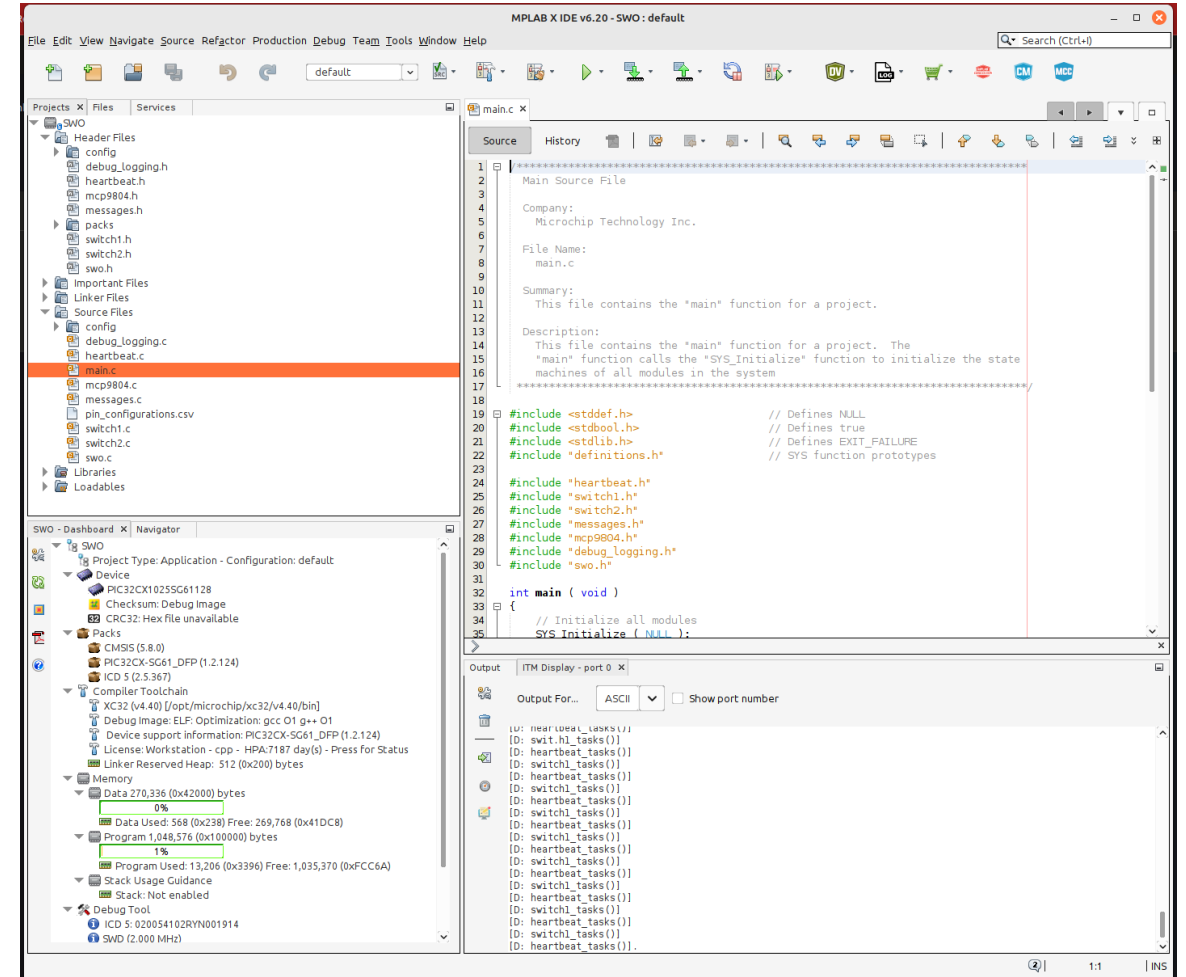
Faults

Fault Types

- **What are they?**
 - **Why are they difficult?**
- **Hard Fault (3)**
 - **Memory Manager (4)**
 - **Bus (5)**
 - **Usage (6)**

Trace back to source

- Decode Fault
- Unwind Stack
- addr2line tool



Fault Entry Behavior

- Context state is saved by the hardware

Stack Frame	Description
SP + 0x1C	xPSR
SP + 0x18	ReturnAddress (PC)
SP + 0x14	LR (R14)
SP + 0x10	R12
SP + 0x0C	R3
SP + 0x08	R2
SP + 0x04	R1
SP + 0x00	R0

xc32-addr2line

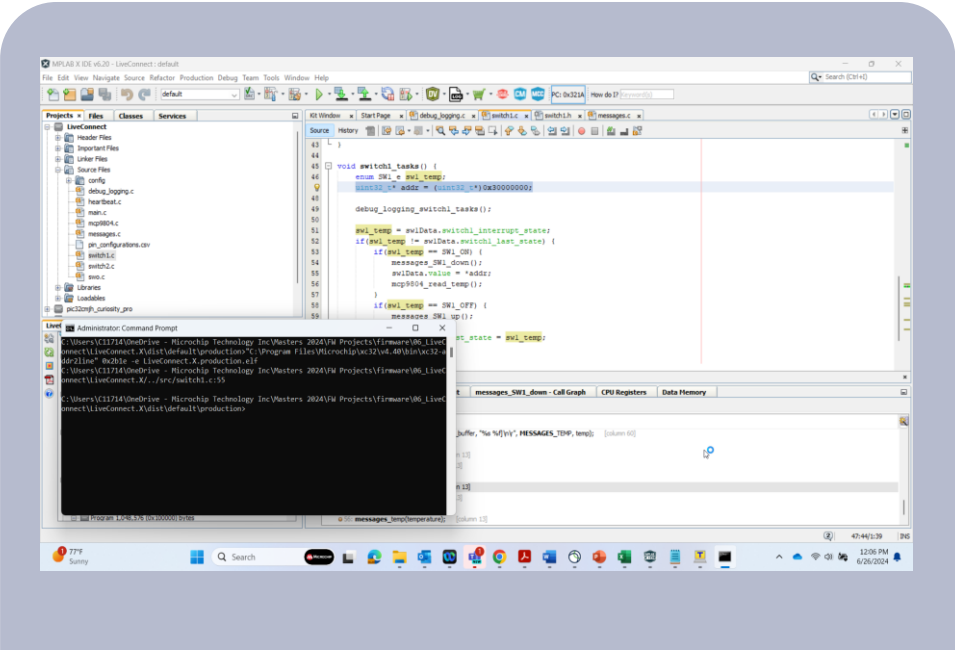
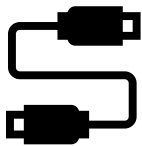
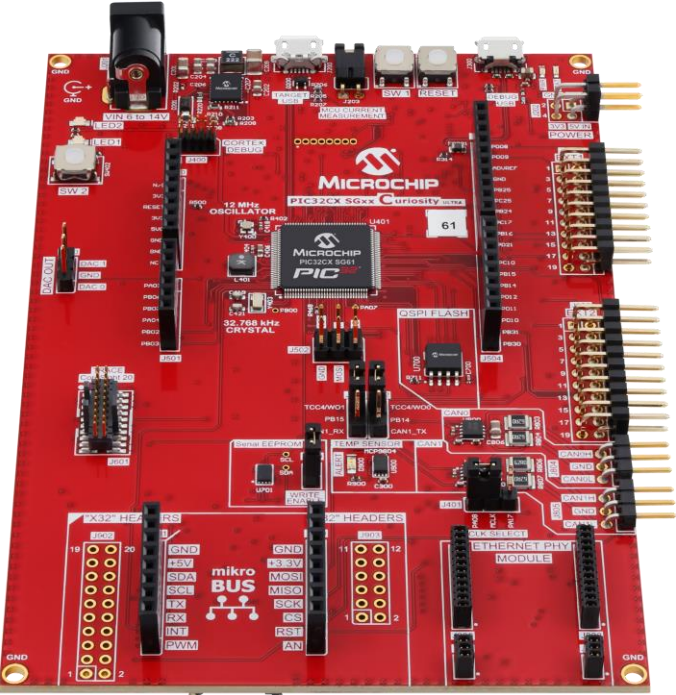
- Command Line tool included with all versions of xc32
- Used with an elf file to translate an address to a C source file and line
- Run with elf file and addresses are supplied via stdin

```
c14029@WAS-LL-C14029: ~/Projects/AdvancedDebugging/firmware/06_LiveConnectDemo/LiveConnectDemo.X/dist/default/production
File Edit View Search Terminal Help
c14029@WAS-LL-C14029:~/Projects/AdvancedDebugging/firmware/06_LiveConnectDemo/LiveConnectDemo.X/dist/default/production$ ls -l
total 451
-rwxrwxr-x 1 c14029 c14029 388480 Jul  9 14:22 LiveConnectDemo.X.production.elf
-rw-rw-r-- 1 c14029 c14029  40152 Jul  9 14:22 LiveConnectDemo.X.production.hex
-rw-rw-r-- 1 c14029 c14029 299934 Jul  9 14:22 LiveConnectDemo.X.production.map
-rwxrwxr-x 1 c14029 c14029 388288 Jul  8 21:03 LiveConnect.X.production.elf
-rw-rw-r-- 1 c14029 c14029  40152 Jul  8 21:03 LiveConnect.X.production.hex
-rw-rw-r-- 1 c14029 c14029 299926 Jul  8 21:03 LiveConnect.X.production.map
-rw-rw-r-- 1 c14029 c14029    452 Jul  9 14:22 memoryfile.xml
c14029@WAS-LL-C14029:~/Projects/AdvancedDebugging/firmware/06_LiveConnectDemo/LiveConnectDemo.X/dist/default/production$ xc32-addr2line -e
LiveConnectDemo.X.production.elf
321a
/home/c14029/Projects/AdvancedDebugging/firmware/06_LiveConnectDemo/LiveConnectDemo.X/../../src/config/default/exceptions.c:106
█
```

Trace Back to Source Demo

- 1. We will connect a PIC32CX SG61 Curiosity Ultra and an ICD5 to the PC, program the application, and press SW1 to cause a fault.**
- 2. We demonstrate the techniques discussed.**
- 3. We will determine the cause and location cause of the fault.**

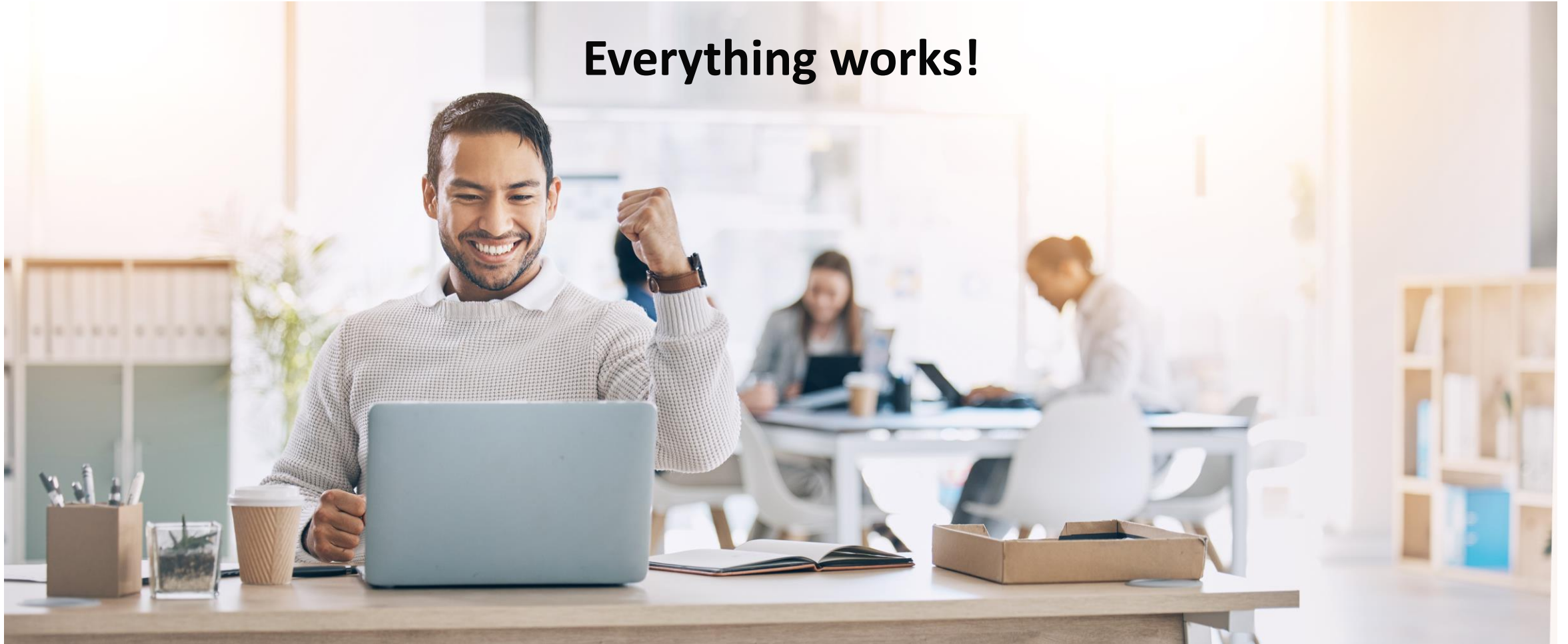
Track Back to Source Demo



PIC32CX SG61 Curiosity Ultra board

Final Run

Everything works!



Adventures in Debugging

What was Learned



Using debug logging for internal state



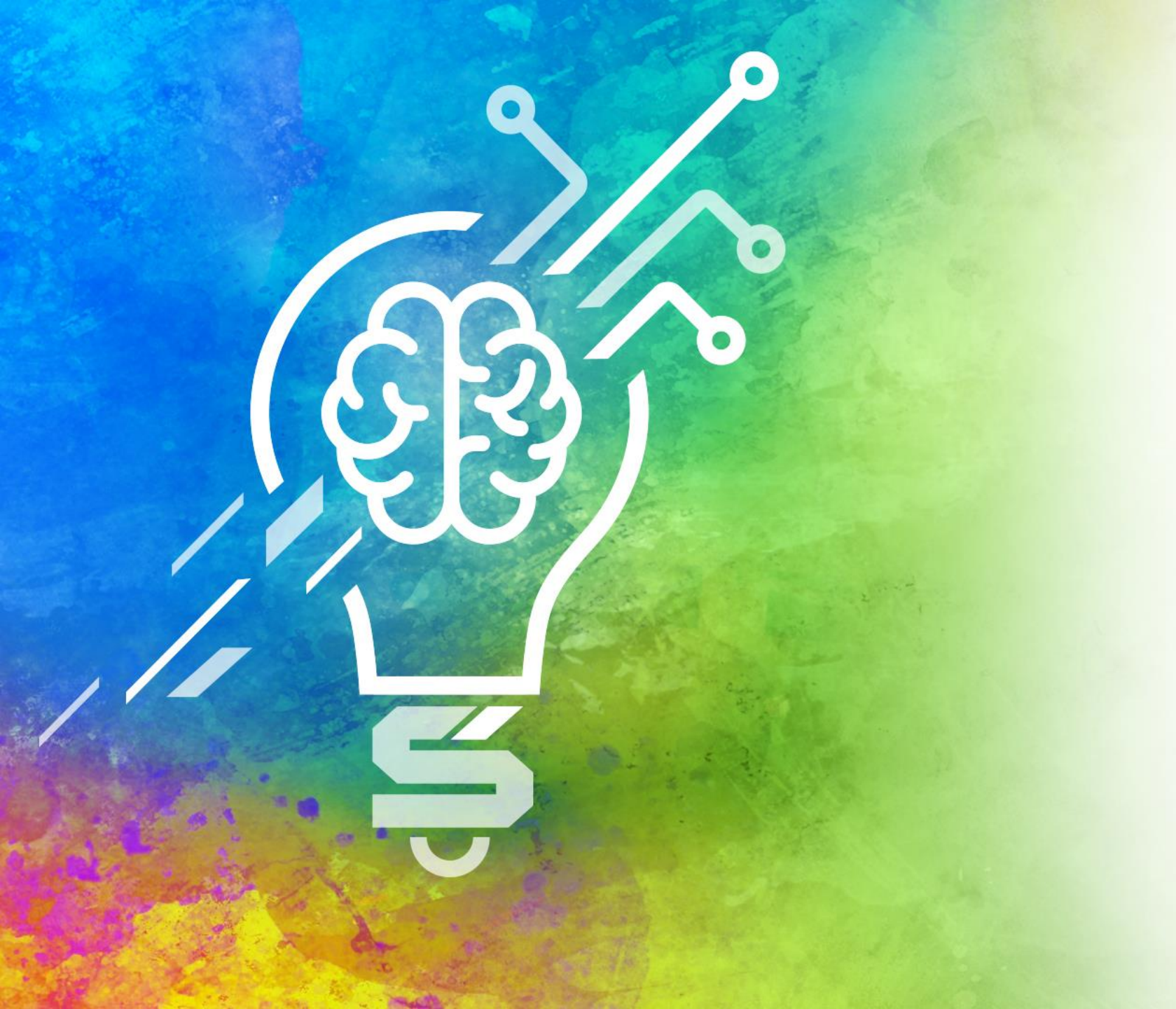
Multiple methods for viewing debug logging and internal state



How to connect to a live target



Tracing back to source using the stack and other tools



Thank You!

LEGAL NOTICE

MASTERS 2024

SOFTWARE:

Microchip software provided to you during the MASTERS 2024 conference is provided “AS IS”, without warranty. Microchip assumes no liability or responsibility for your use of any Microchip software. You must use Microchip software exclusively with Microchip products. Further, use of Microchip software is subject to copyright and the disclaimers and license terms accompanying such software, whether set forth at the install of each program or posted in a header or text file.

Notwithstanding the above, certain components of software offered by Microchip and 3rd parties may be covered by “open source” software licenses, including licenses that require that the distributor of such software make the software available in source code format. To the extent required by any such open source software license, the terms of such license govern.

Any transfer of Microchip product, technology, or software may be subject to export controls. Microchip encourages any entity transferring product, technology, or software to seek appropriate legal advice and/or consult the applicable governmental agencies prior to transfer. The information provided herein is subject to change at any time and without notice.

NOTICE & DISCLAIMER:

Materials, software and accompanying information (including, for example, any references to 3rd party companies and 3rd party websites) are for informational purposes only and are provided “AS IS.” Microchip assumes no responsibility for statements made by 3rd party companies, or materials or information that such 3rd parties may provide.

MICROCHIP DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, OR STATUTORY, INCLUDING ANY IMPLIED WARRANTIES OF NONINFRINGEMENT, MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY DIRECT OR INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND RELATED TO THESE MATERIALS OR ACCOMPANYING INFORMATION PROVIDED TO YOU BY MICROCHIP OR OTHER THIRD PARTIES, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR THE DAMAGES ARE FORESEEABLE. PLEASE BE AWARE THAT THE IMPLEMENTATION OF INTELLECTUAL PROPERTY PRESENTED IN MASTERS 2024 MAY REQUIRE A LICENSE FROM MICROCHIP OR THIRD PARTIES.

TRADEMARKS:

The Microchip name and logo and the Microchip logo are registered trademarks of Microchip Technology Incorporated in the U.S.A. and other countries. Information on and a listing of Microchip trademarks is available at <https://www.microchip.com/en-us/about/legal-information/microchip-trademarks>.

All other trademarks mentioned herein are property of their respective companies.

COPYRIGHT:

© 2024 Microchip Technology Incorporated, All Rights Reserved.